

INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN: AN AGILE, ITERATIVE APPROACH

SATZINGER | JACKSON | BURD

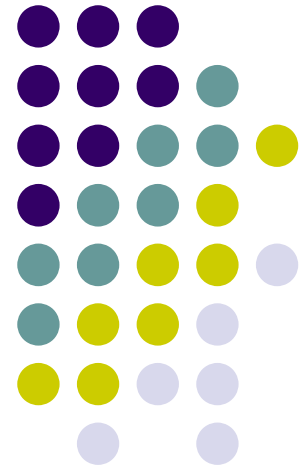
CHAPTER 5

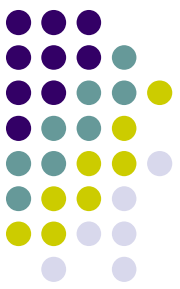
Extending the Requirements Models

Chapter 5

Introduction to Systems
Analysis and Design:
An Agile, Interactive Approach
6th Ed

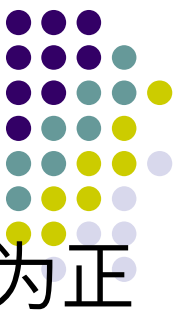
Satzinger, Jackson & Burd





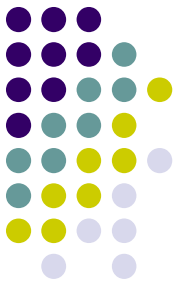
Chapter 5 Outline

- Use Case Descriptions (用例描述)
- Activity Diagrams (活动图) for Use Cases
- The System Sequence Diagram (系统顺序图) — Identifying Inputs and Outputs
- The State Machine Diagram (状态机图) — Identifying Object Behavior
- Integrating Requirements Models (需求模型的集成)



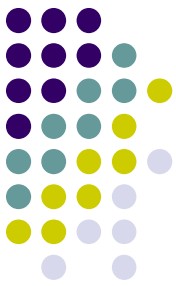
Learning Objectives

- Write fully developed use case descriptions (为正开发的用例撰写完整的描述)
- Develop activity diagrams to model flow of activities (开发活动图来建立活动流的描述)
- Develop system sequence diagrams (开发系统顺序图)
- Develop state machine diagrams to model object behavior (开发状态机图为对象行为建模)
- Explain how use case descriptions and UML diagrams work together to define functional requirements (解释如何结合用例描述定义功能需求)



Chapter 5 Outline

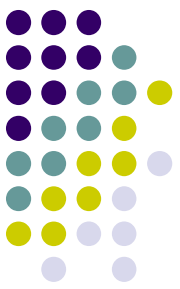
- Use Case Descriptions (用例描述)
- Activity Diagrams for Use Cases
- The System Sequence Diagram—Identifying Inputs and Outputs
- The State Machine Diagram—Identifying Object Behavior
- Integrating Requirements Models



Use Case Descriptions

- Write a *brief description* as shown in Chapter 3 for most use cases.

Use case	Brief use case description
<i>Create customer account</i>	User/actor <u>enters new customer account data, and the system assigns account number, creates a customer record, and creates an account record.</u>
<i>Look up customer</i>	User/actor enters customer account number, and the system retrieves and displays customer and account data.
<i>Process account adjustment</i>	User/actor enters order number, and the system retrieves customer and order data; actor enters adjustment amount, and the system creates a transaction record for the adjustment.



Use Case Descriptions

- Write a *fully developed use case description* for more complex use cases (为更复杂的用例编写一个完全开发的用例描述)
- **Typical use case description templates include:**
 - Use case name (用例名称)
 - Scenario (if needed) (场景)
 - Triggering event (触发事件)
 - Brief description (简要描述)
 - Actors (相关人员)
 - Related use cases (<<includes>>) (相关用例)
 - Stakeholders (利益相关者)
 - Preconditions (先决条件)
 - Post conditions (后置条件)
 - Flow of activities (活动流程)
 - Exception conditions (异常条件)

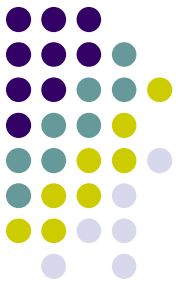
Fully Developed Use Case Description

Use case: Create customer account (创建客户账户)

Use case name:	Create customer account.	
Scenario:	Create online customer account.	
Triggering event:	New customer wants to set up account online.	
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.	
Actors:	Customer.	
Related use cases:	Might be invoked by the <i>Check out shopping cart</i> use case.	
Stakeholders:	Accounting, Marketing, Sales.	
Preconditions:	Customer account subsystem must be available. Credit/debit authorization services must be available.	
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.	
Flow of activities:	Actor	System
	1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer. 1.2 System prompts for customer addresses.
	2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.
	3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.
Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.	

Fully Developed Use Case Description

Create customer account (part 1)



用例名	创建客户账户
场景	创建在线客户账户
触发事件	新用户想要新账户
相关人员	客户
相关用例	可能由结账购物车用例调用
利益相关者	会计，市场影响，销售
前置条件	客户账户子系统应当可用； 信用卡 / 借记卡授权服务必须可用
后置条件	必须创建并保存客户； 必须创建并保存一个或多个地址； 信用卡 / 借记卡信息必须经过验证； 必须创建并保存帐户； 地址和帐户必须与客户相关联

Fully Developed Use Case Description

Use case: *Create customer account*

Use case name:	Create customer account.	
Scenario:	Create online customer account.	
Triggering event:	New customer wants to set up account online.	
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.	
Actors:	Customer.	
Related use cases:	Might be invoked by the <i>Check out shopping cart</i> use case.	
Stakeholders:	Accounting, Marketing, Sales.	
Preconditions:	Customer account subsystem must be available. Credit/debit authorization services must be available.	
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.	
Flow of activities:	Actor	System
	1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer. 1.2 System prompts for customer addresses.
	2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.
	3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.
Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.	

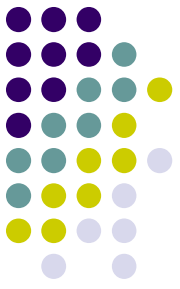
Fully Developed Use Case Description

Create customer account (part 2)



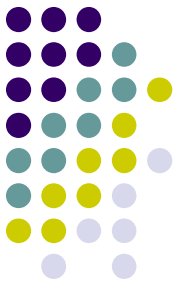
活动流程	用户	系统
	1. 客户想要创建客户账号并输入了基本客户信息	1.1 系统创建新客户 1.2 系统需要客户的地址
	2. 客户输入一个或多个地址	2.1 系统创建地址 2.2 系统需要客户的信用卡信息
	3. 客户输入信用卡信息	3.1 系统创建账户 3.2 系统验证信用卡授权 3.3 系统关联客户，地址和账户 3.4 系统返回合法用户账户细节
异常条件	1.1 基本客户信息不完整 2.1 地址不合法 3.2 信用卡信息不合法	

Use Case Description Details



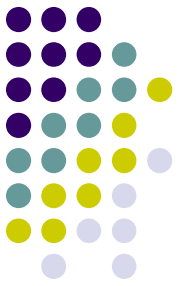
- Use case name (用例名称)
 - Verb-noun (动词 - 名词)
- Scenario (if needed) (场景)
 - A use case can have more than one scenario (special case or more specific path) 一个用例可以有多个场景
- Triggering event (触发事件)
 - Based on event decomposition technique (基于事件分解技术)
- Brief description (简要描述)
 - Written previously when use case was identified (在确定用例时编写)
- Actors (用户)
 - One or more users from use case diagrams (来自用例图的一个或多个用户)

Use Case Description Details



- Related use cases <<includes>> 相关用例
 - If one use case invokes or includes another (如果一个用例调用或包含另一个用例)
- Stakeholders (利益相关者)
 - Anyone with an interest in the use case (任何对用例感兴趣的人)
- Preconditions (先决条件)
 - What must be true before the use case begins (在用例开始之前, 什么必须是正确的)
- Post conditions (后置条件)
 - What must be true when the use case is completed (当用例完成时, 什么必须是正确的)
 - Use for planning test case expected results (用于计划测试用例的预期结果)

Use Case Description Details



- Flow of activities (活动流程)
 - The activities that go on between actor and the system (参与者和系统之间进行的活动)
- Exception conditions (异常条件)
 - Where and what can go wrong (哪里和什么地方会出错)

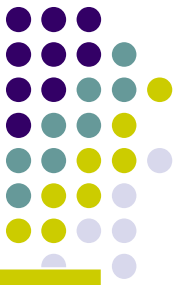
Another Fully Developed Use Case Description Example

Use case *Ship items* (运输物品)

Use case name:	Ship items.	
Scenario:	Ship items for a new sale.	
Triggering event:	Shipping is notified of a new sale to be shipped.	
Brief description:	Shipping retrieves sale details, finds each item and records it is shipped, records which items are not available, and sends shipment.	
Actors:	Shipping clerk.	
Related use cases	None.	
Stakeholders:	Sales, Marketing, Shipping, warehouse manager.	
Preconditions:	Customer and address must exist. Sale must exist. Sale items must exist.	
Postconditions:	Shipment is created and associated with shipper. Shipped sale items are updated as shipped and associated with the shipment. Unshipped items are marked as on back order. Shipping label is verified and produced.	
Flow of activities:	Actor	System
	1. Shipping requests sale and sale item information.	1.1 System looks up sale and returns customer, address, sale, and sales item information.
	2. Shipping assigns shipper.	2.1 System creates shipment and associates it with the shipper.
	3. For each available item, shipping records item is shipped.	3.1 System updates sale item as shipped and associates it with shipment.
	4. For each unavailable item, shipping records back order.	4.1 System updates sale item as on back order.
	5. Shipping requests shipping label supplying package size and weight.	5.1 System produces shipping label for shipment. 5.2 System records shipment cost.
Exception conditions:	2.1 Shipper is not available to that location, so select another. 3.1 If order item is damaged, get new item and updated item quantity. 3.1 If item bar code isn't scanning, shipping must enter bar code manually. 5.1 If printing label isn't printing correctly, the label must be addressed manually.	

Fully Developed Use Case Description

Ship items (part 1)



用例名	运输物品
场景	为一个新的订单运输物品
触发事件	运输新订单
简短描述	运输订单，找到对应物品并记录这些物品被运输，记录哪些物品缺货，送出运输
用户	发货员
相关用例	无
利益相关者	销售，市场营销，运输，仓库管理
前置状态	客户和地址必须存在；订单必须存在，订单物品必须存在
后置状态	运输由运输者创建和关联； 运输的订单物品会跟着运输状态而更新； 未运输的物品应当被标记为缺货预定； 运输标签被确认并生成

Another Fully Developed Use Case Description Example

Use case *Ship items*

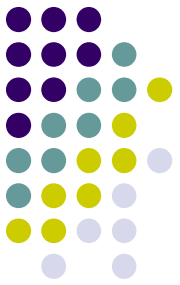
Use case name:	Ship items.	
Scenario:	Ship items for a new sale.	
Triggering event:	Shipping is notified of a new sale to be shipped.	
Brief description:	Shipping retrieves sale details, finds each item and records it is shipped, records which items are not available, and sends shipment.	
Actors:	Shipping clerk.	
Related use cases	None.	
Stakeholders:	Sales, Marketing, Shipping, warehouse manager.	
Preconditions:	Customer and address must exist. Sale must exist. Sale items must exist.	
Postconditions:	Shipment is created and associated with shipper. Shipped sale items are updated as shipped and associated with the shipment. Unshipped items are marked as on back order. Shipping label is verified and produced.	
Flow of activities:	Actor	System
	1. Shipping requests sale and sale item information.	1.1 System looks up sale and returns customer, address, sale, and sales item information.
	2. Shipping assigns shipper.	2.1 System creates shipment and associates it with the shipper.
	3. For each available item, shipping records item is shipped.	3.1 System updates sale item as shipped and associates it with shipment.
	4. For each unavailable item, shipping records back order.	4.1 System updates sale item as on back order.
	5. Shipping requests shipping label supplying package size and weight.	5.1 System produces shipping label for shipment. 5.2 System records shipment cost.
Exception conditions:	2.1 Shipper is not available to that location, so select another. 3.1 If order item is damaged, get new item and updated item quantity. 3.1 If item bar code isn't scanning, shipping must enter bar code manually. 5.1 If printing label isn't printing correctly, the label must be addressed manually.	

Fully Developed Use Case Description

Create customer account (part 2)



活动流程	用户	系统
	1. 请求订单和订单物品的信息	1.1 系统查到订单并且返回客户，地址，订单，和订单信息
	2. 为运输分配运输人员	2.1 系统创建运输并关联相关运输人员
	3. 对于每一个有货的物品，更新他们的运输信息为已运输	3.1 系统更新订单物品为已运输并关联到运输信息
	4. 对每一个缺货的物品，记录为缺货订购订单	4.1 系统更新订单物品为缺货订单
	5. 运输请求运输标签，标签上提供包裹大小和重量	5.1 系统生成运输标签 5.2 系统记录运输费用
异常条件	2.1 运输人员不在该位置，需要选择另一个人员 3.1 如果订单物品被损坏，取一个新的物品并且更新物品数量 3.1 如果物品二维码无法正确扫描，需要手动输入二维码 5.1 输入打印的标签不正确，标签需要能够手动修改	



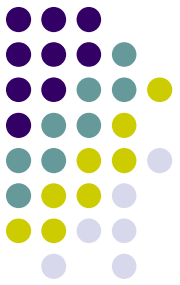
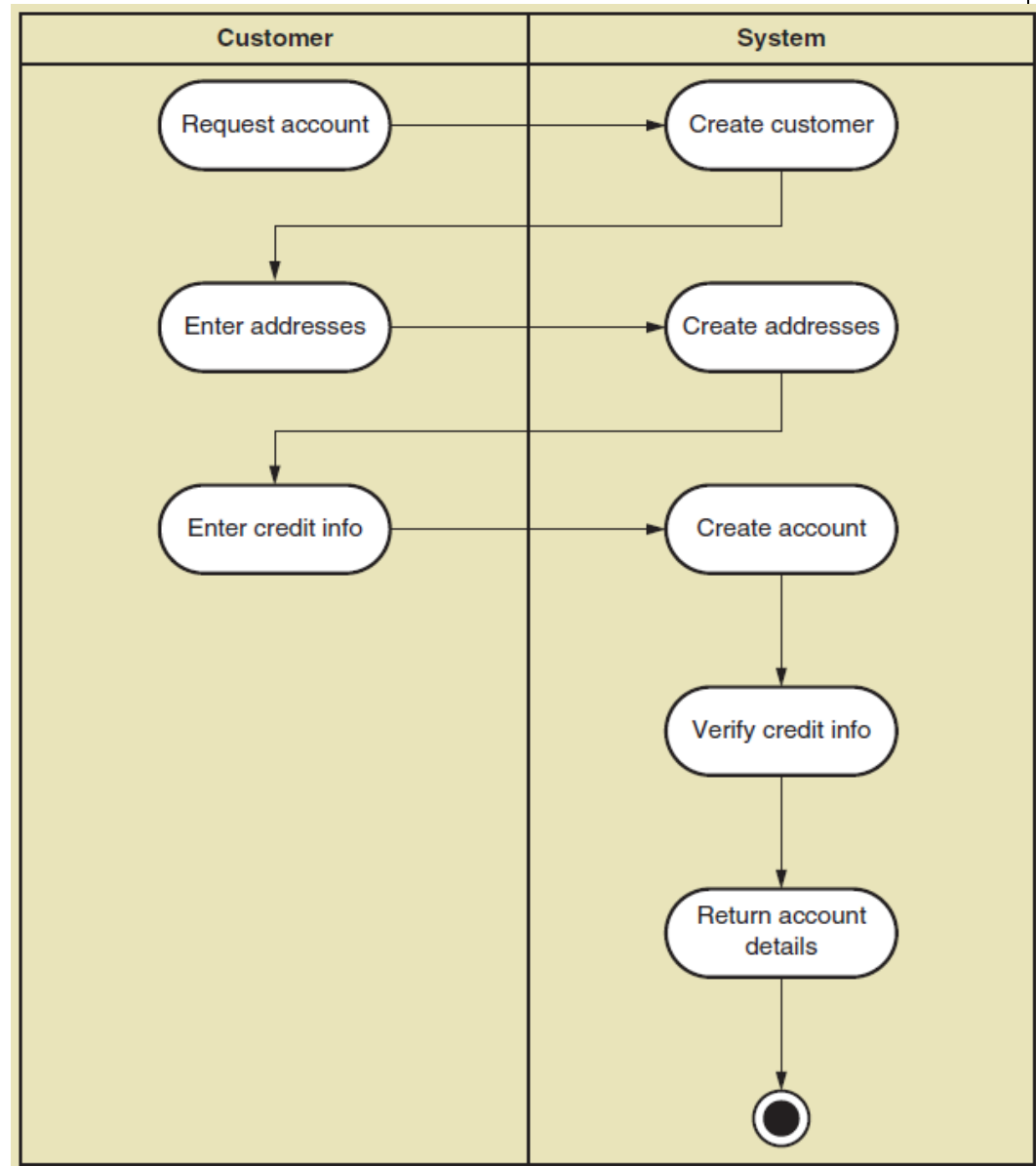
Chapter 5 Outline

- Use Case Descriptions
- Activity Diagrams for Use Cases (用例活动图)
- The System Sequence Diagram—Identifying Inputs and Outputs
- The State Machine Diagram—Identifying Object Behavior
- Integrating Requirements Models

UML Activity Diagram for Use Case

Create Customer Account (创建客户账户)

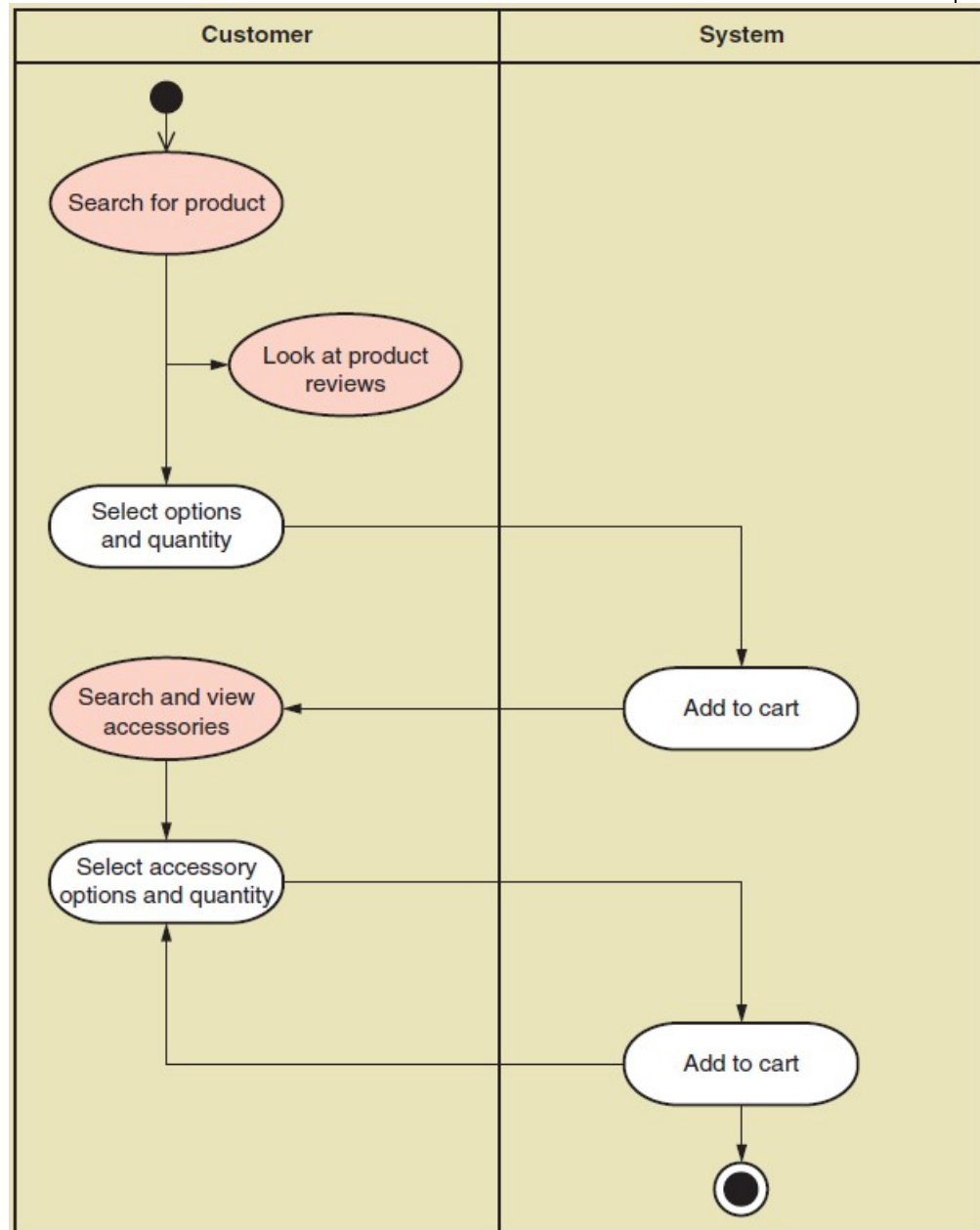
Note: this
shows flow of
activities only

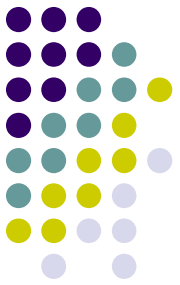


UML Activity Diagram for Use Case

Fill shopping cart (添加购物车)

Note: this shows use case with <<includes>> relationship

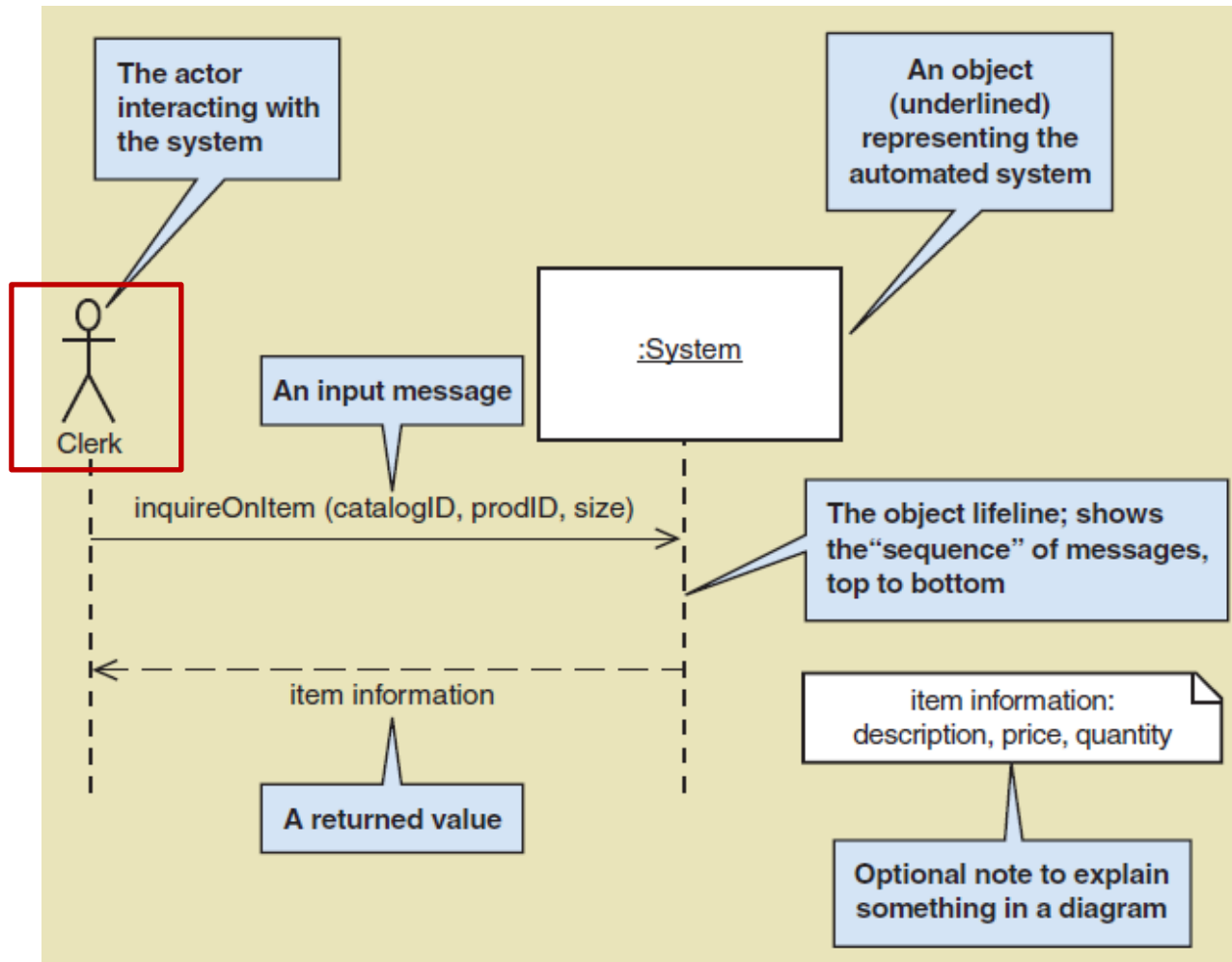
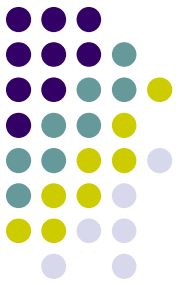




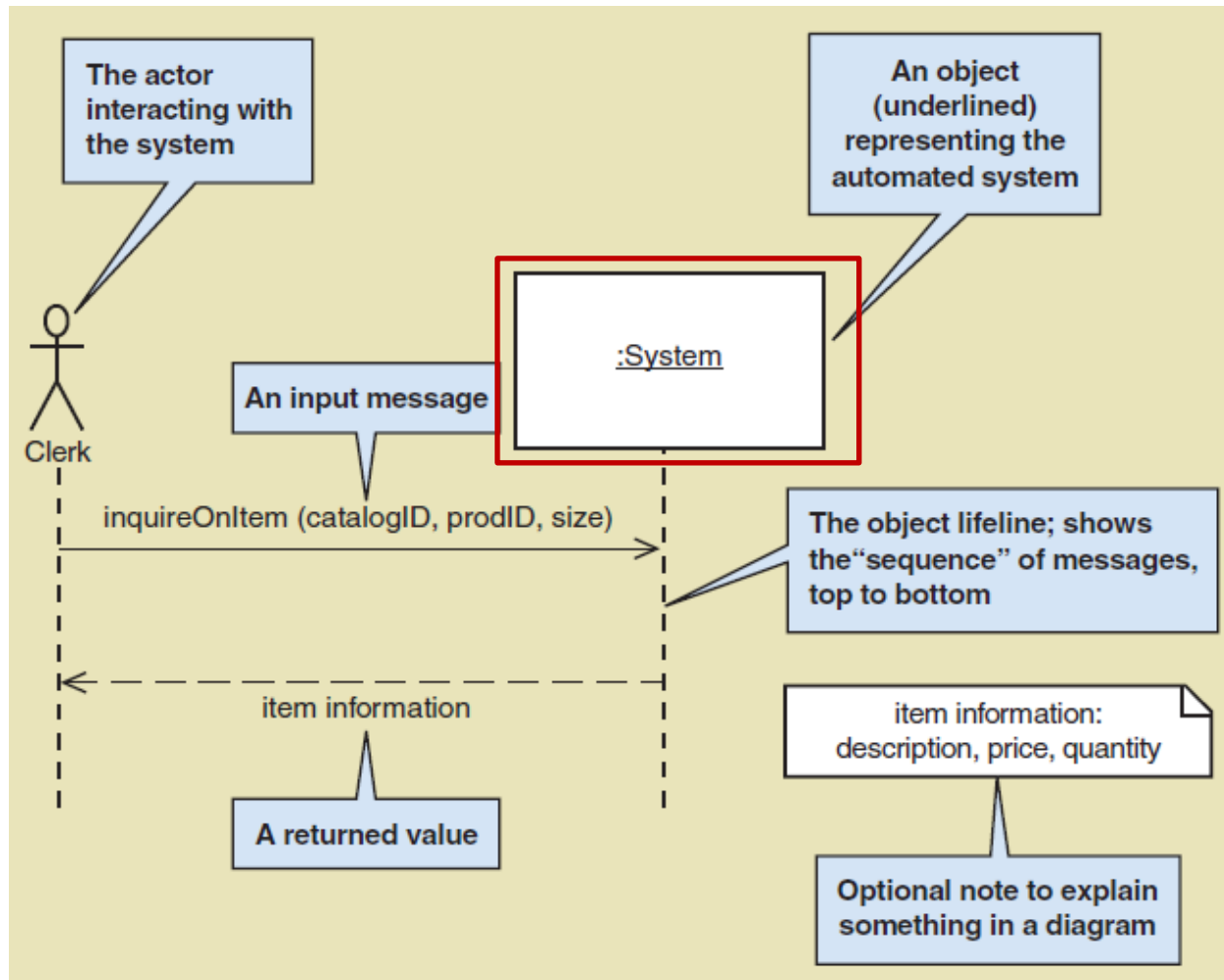
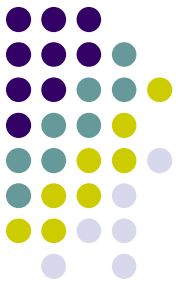
Chapter 5 Outline

- Use Case Descriptions
- Activity Diagrams for Use Cases
- The System Sequence Diagram—Identifying Inputs and Outputs （系统顺序图 - 识别输入和输出）
- The State Machine Diagram—Identifying Object Behavior
- Integrating Requirements Models

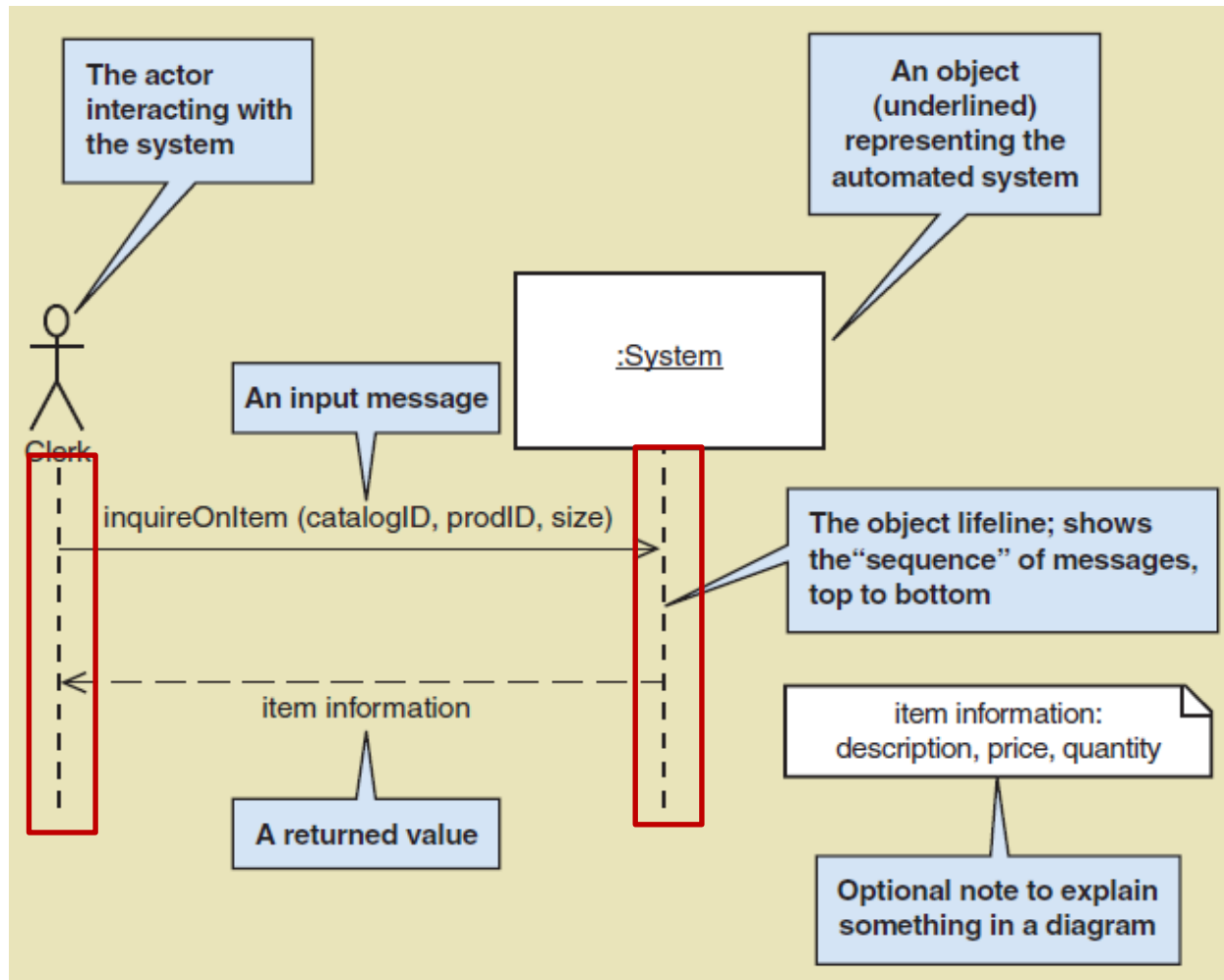
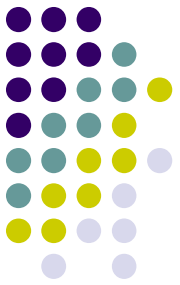
System Sequence Diagram (SSD) Notation



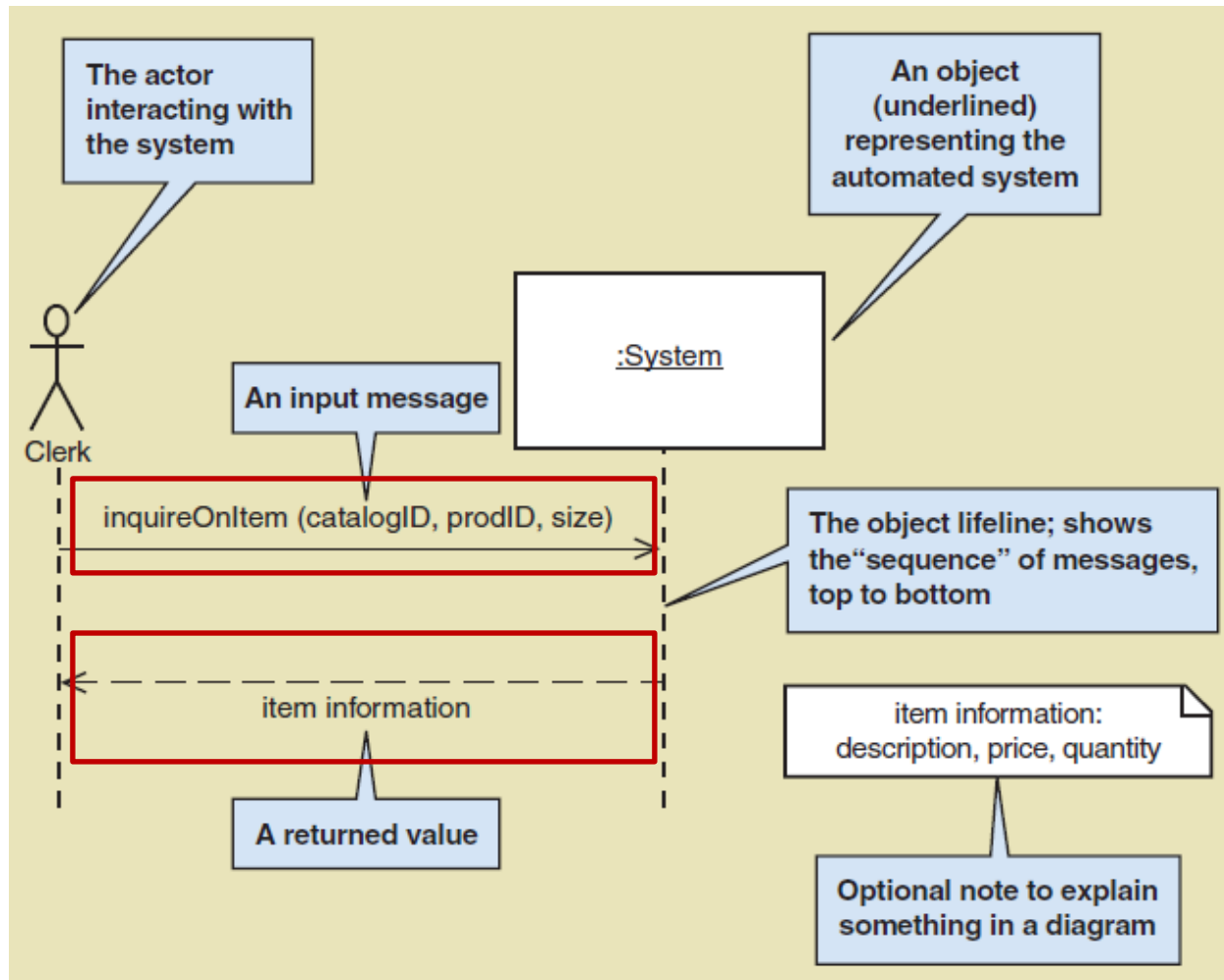
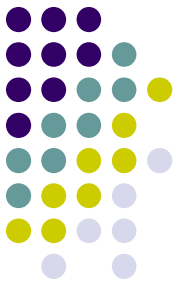
System Sequence Diagram (SSD) Notation



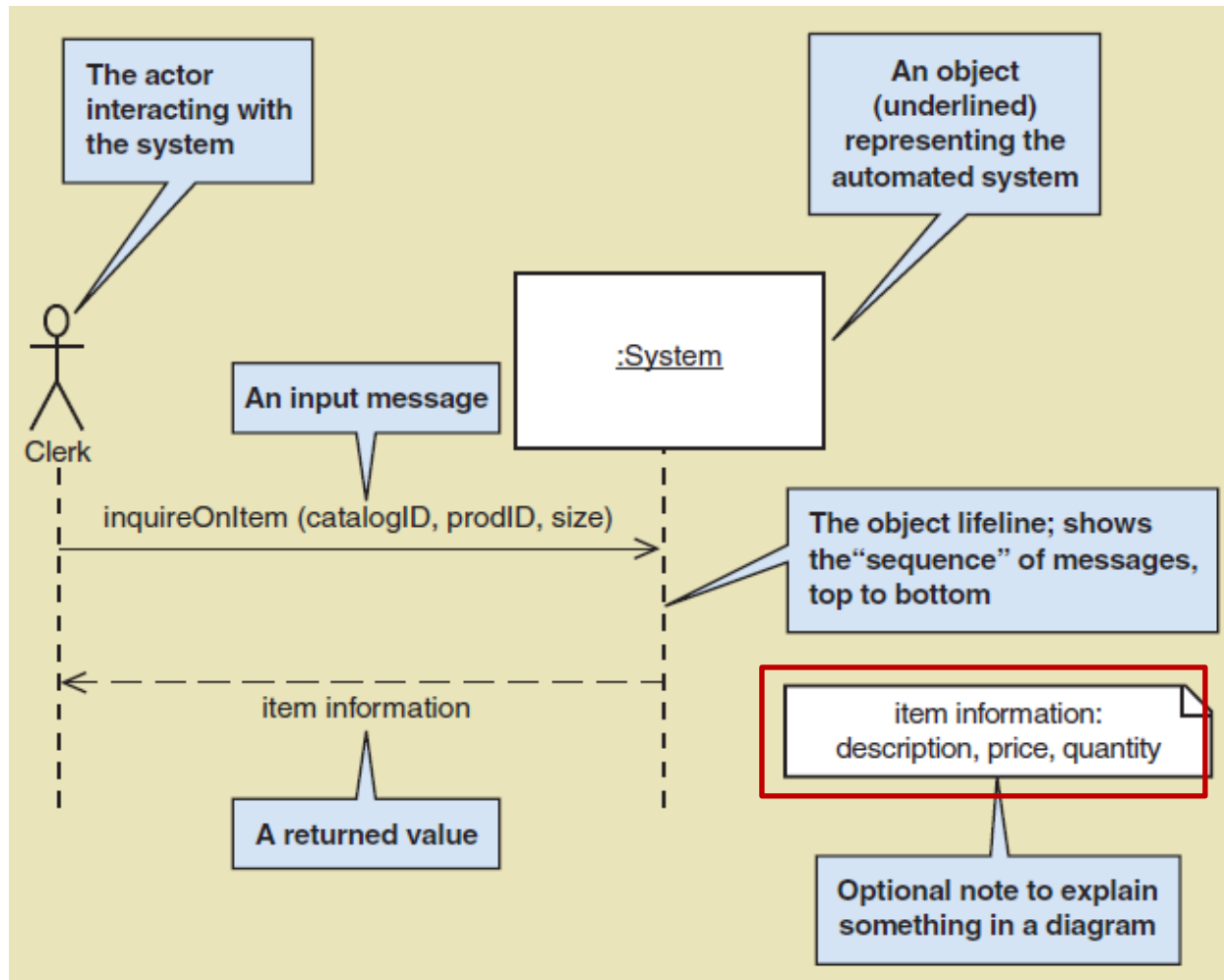
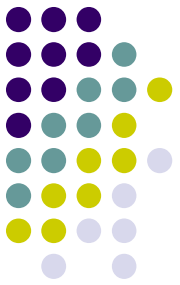
System Sequence Diagram (SSD) Notation



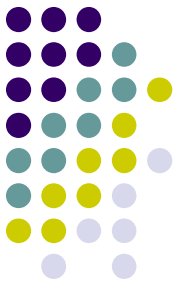
System Sequence Diagram (SSD) Notation



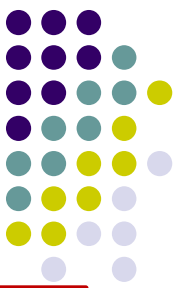
System Sequence Diagram (SSD) Notation



System Sequence Diagram (SSD)



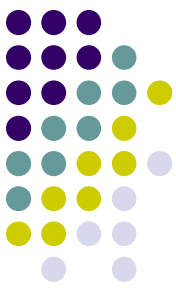
- A UML sequence diagram (UML 顺序图)
- Special case for a sequence diagram (顺序图的特殊情况)
 - Only shows actor and one object (只显示人员和一个对象)
 - The one object represents the complete system (一个对象代表整个系统)
 - Shows input & output messaging requirements for a use case (显示用例的输入和输出消息来传递需求)
- Actor, :System, object lifeline
- Messages



Message Notation

** [true/false condition] return-value := message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
- Brackets [] indicate a true/false condition. This is a test for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
- Message-name is the description of the requested service. It is omitted on dashed-line return messages, which only show the return data parameters.
- Parameter-list (with parentheses on initiating messages and without parentheses on return messages) shows the data that are passed with the message.
- Return-value on the same line as the message (requires :=) is used to describe data being returned from the destination object to the source object in response to the message.

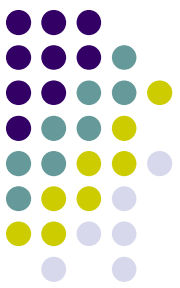


Message Notation

***** *[true/false condition] return-value := message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
- Brackets [true/false condition] are optional. This is used for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
- Message-name is the description of the requested service. It is omitted on dashed-line return messages, which only show the return data parameters.
- Parameter-list (with parentheses on initiating messages and without parentheses on return messages) shows the data that are passed with the message.
- Return-value on the same line as the message (requires :=) is used to describe data being returned from the destination object to the source object in response to the message.

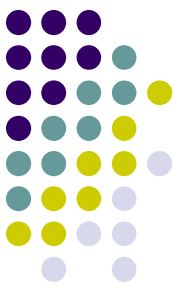
星号：消息是重复的或者循环发生的



Message Notation

* *[true/false condition] return-value := message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
- Brackets [] indicate a true/false condition. This is a test for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
- Message-name is the description of the requested service. It is omitted on dashed-**括号：真 / 假的状态判断；如果为真，则消息发送，否则不发送** return data parameters.
- Parameter list (shown in parentheses on return messages) shows the data that are passed with the message.
- Return-value on the same line as the message (requires :=) is used to describe data being returned from the destination object to the source object in response to the message.



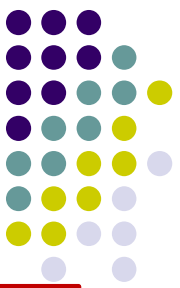
Message Notation

* *[true/false condition] return-value := message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
- Brackets [] indicate a true/false condition. This is a test for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
- Message-name is the description of the requested service. It is omitted on dashed-line return messages, which only show the return data parameters.
- Parameter-list (with parentheses on initiating messages and without parentheses on dashed-line return messages).
- Return-value on the same line as the message (requires :=) is used to describe data being returned from the destination object to the source object in response to the message.

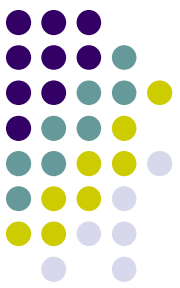
消息名：对所请求服务的描述。它在虚线返回消息中被省略，虚线返回消息只显示返回数据参数

Message Notation



* *[true/false condition] return-value := message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
 - Brackets [] indicate a true/false condition. This is a test for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
 - Message-name is the description of the requested service. It is omitted on dashed-line return messages, which only show the return data parameters.
 - Parameter-list (with parentheses on initiating messages and without parentheses on return messages) shows the data that are passed with the message.
 - Return-value on the same line as the message (requires :=) is used to describe the return value of the message.
- 参数列表：显示与消息一起传递的数据（初始消息有括号，返回消息没有括号）



Message Notation

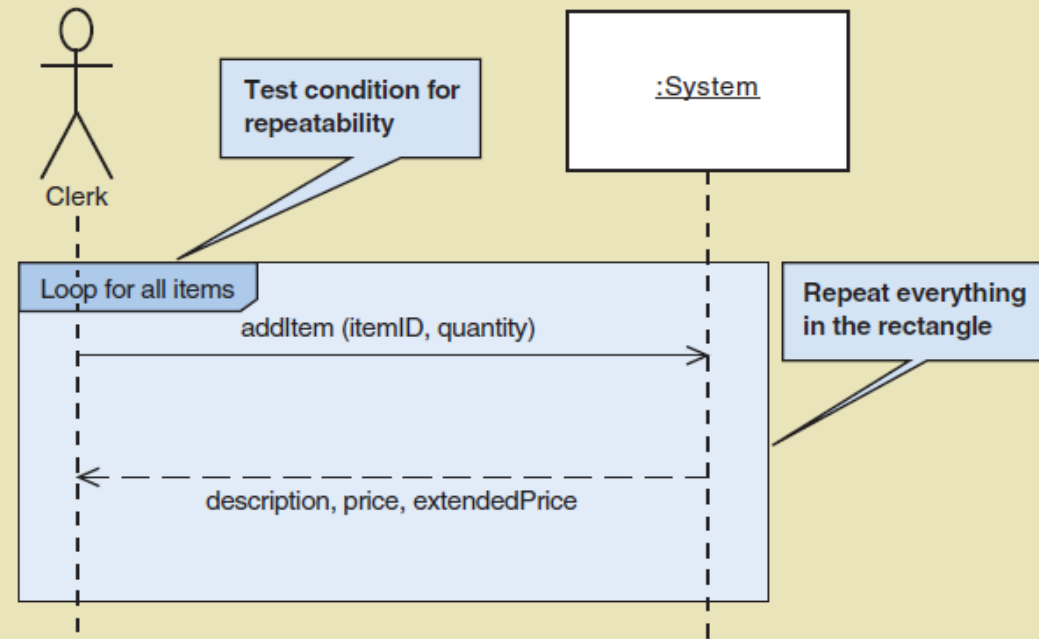
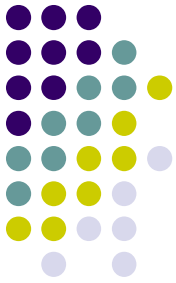
* *[true/false condition]* **return-value** := *message-name (parameter-list)*

- An asterisk (*) indicates repeating or looping of the message.
- Brackets [] indicate a true/false condition. This is a test for that message only. If it evaluates to true, the message is sent. If it evaluates to false, the message isn't sent.
- Message-name is the description of the requested service. It is omitted on dashed-line return messages, which only show the return data parameters.
- Parameter-list (with parentheses on initiating messages and without parentheses on return messages).

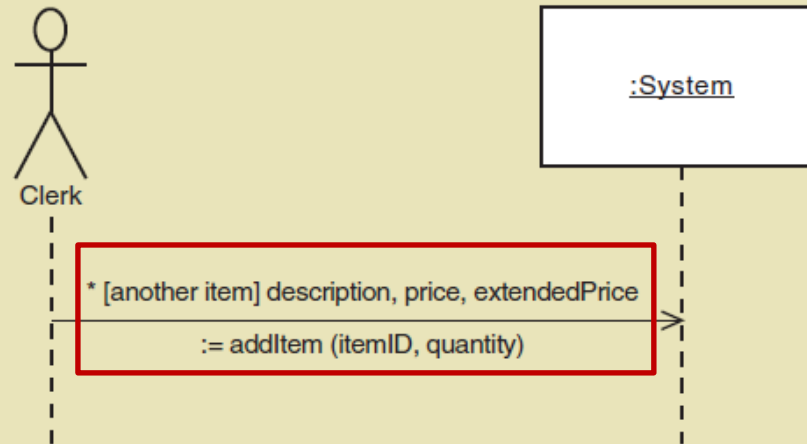
返回值：描述作为对消息的响应从目标对象返回到源对象的数据

- Return-value on the same line as the message (requires :=) is used to describe data being returned from the destination object to the source object in response to the message.

UML Message Examples with Loop Frame (带环路的系 统顺序图)



(a) Detailed notation

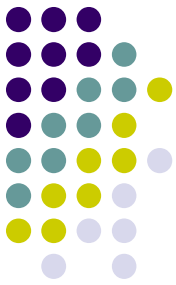
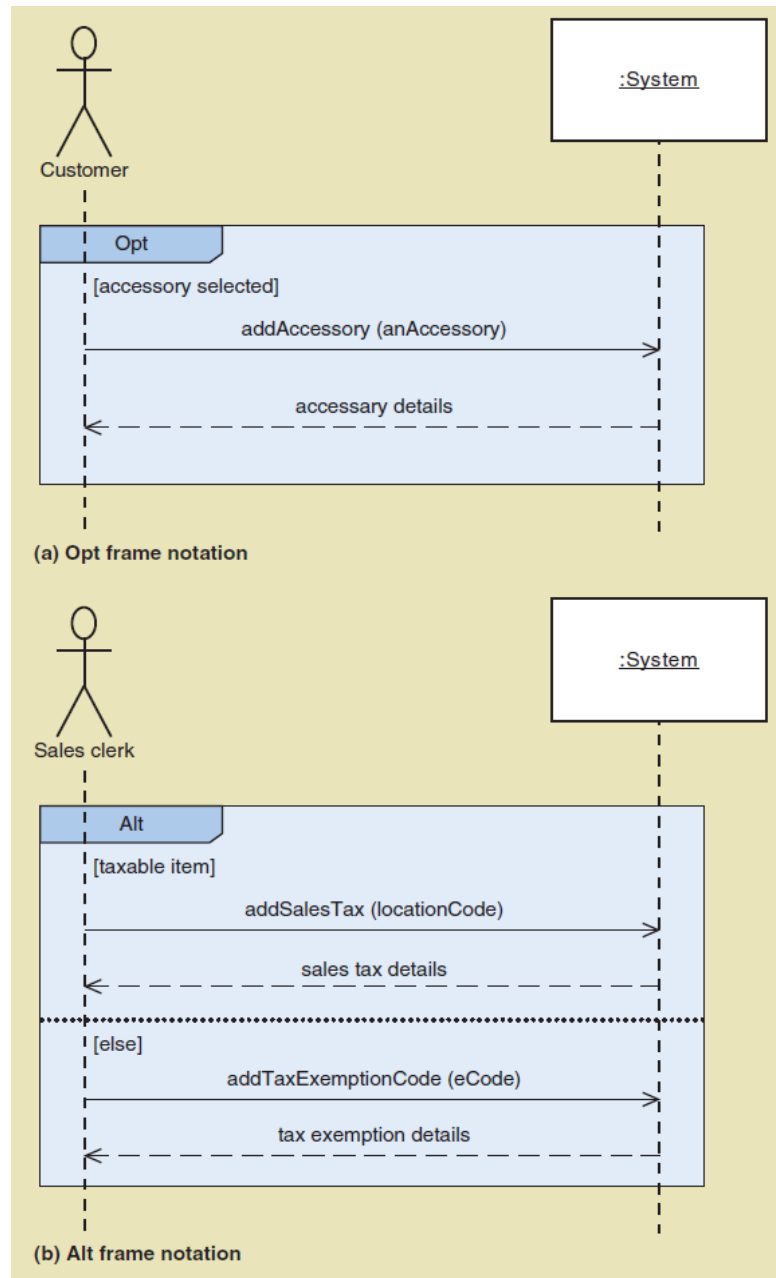


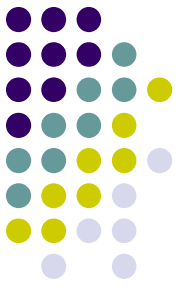
(b) Alternate notation

SSD Message Examples

Opt Frame (optional) 可选框架

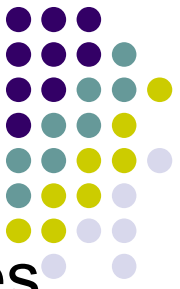
Alt Frame (if-else) 判断框架





Steps for Developing SSD

1. Identify **input** message (识别输入消息)
 - See use case flow of activities or activity diagram (请参阅活动的用例流或活动图)
2. Describe the message from the external actor to the system using the **message** notation (使用消息符号描述从外部参与者到系统的消息)
 - Name it verb-noun: what the system is asked to do (使其命名为动词 - 名词：系统被要求做什么)
 - Consider parameters the system will need (考虑系统需要的参数)

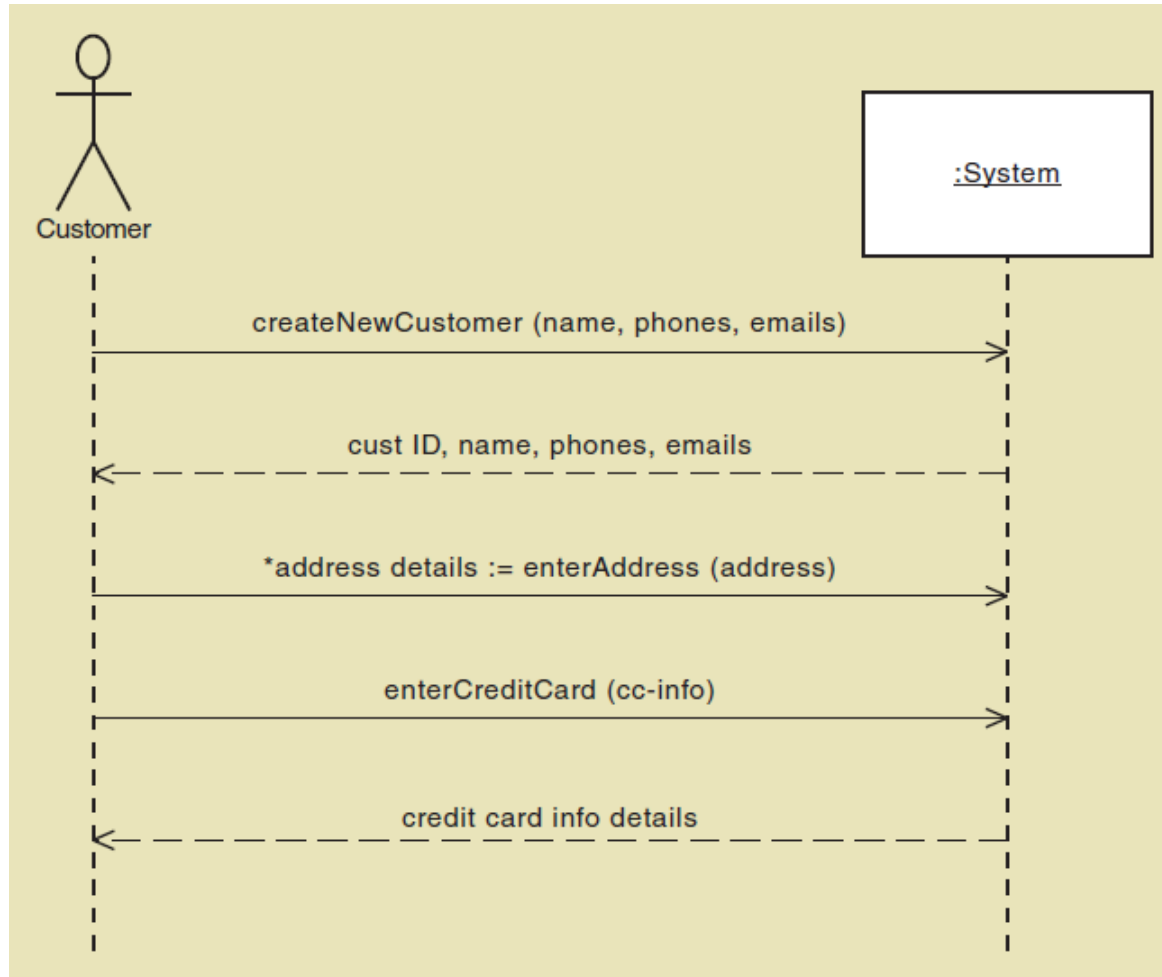
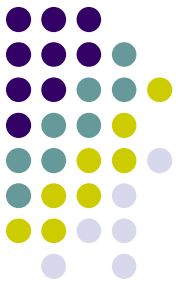


Steps for Developing SSD

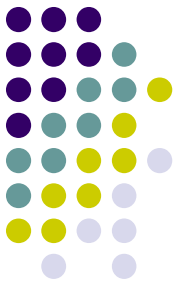
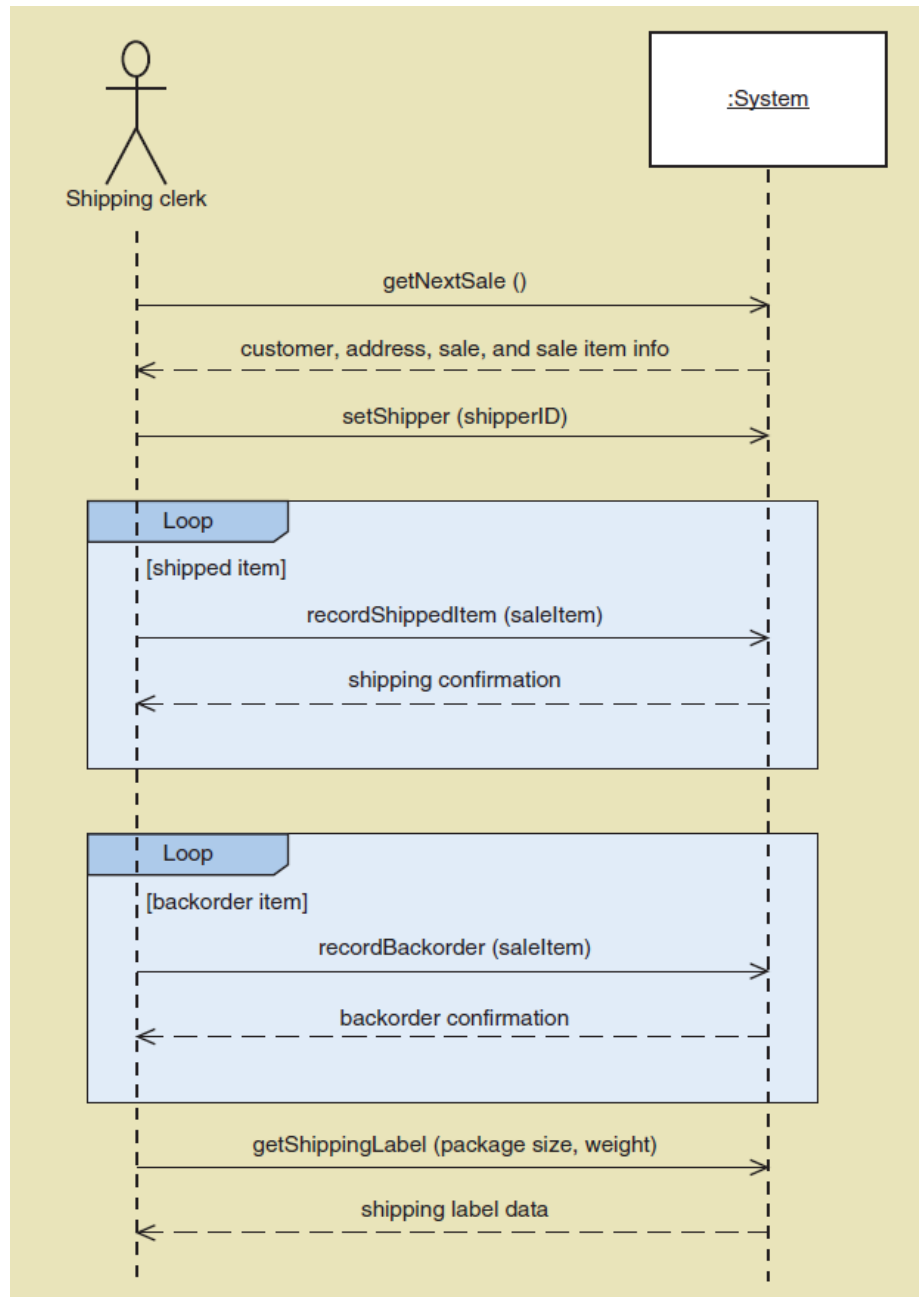
3. Identify any **special conditions** on input messages
(确定输入消息的任何特殊条件)
 - Iteration/loop frame (循环 / 迭代框架)
 - Opt or Alt frame (选择或判断框架)
3. Identify and add **output** return values (识别并添加输出返回值)
 - On message itself: Value:= getValue(valueID) (关于消息本身: value:=getValue(valueID))
 - As explicit return on separate dashed line (在单独的虚线上显式返回)

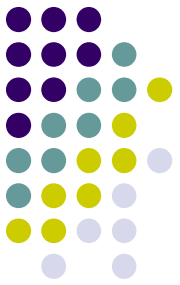
UML for create customer account

Use case （创建客户账户用例的顺序图）



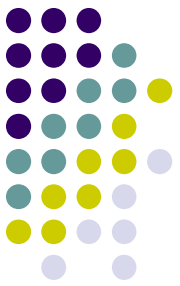
SSD for *Ship items* Use Case (创建运输 商品用例的 顺序图)





Chapter 5 Outline

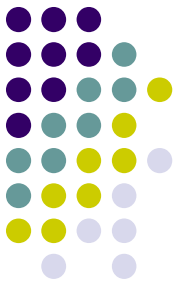
- Use Case Descriptions
- Activity Diagrams for Use Cases
- The System Sequence Diagram—Identifying Inputs and Outputs
- The State Machine Diagram—Identifying Object Behavior (状态机图 - 识别对象行为)
- Integrating Requirements Models



State Machine Diagram

- State machine diagram (状态机图)
 - A UML diagram showing the life of an object in **states** and **transitions** (显示处于**状态**和**转换**中的对象生命周期的 UML 图)
- State (状态; 动名词或动词短语)
 - A condition during an object's life when it satisfies some criterion, performs some action, or waits for an event (对象生命周期中满足某些标准、执行某些操作或等待某个事件的条件)
- Transition (状态转换)
 - The movement of an object from one state to another state (物体从一种状态移动到另一种状态)
- Action Expression (行动表达)
 - A description of activities performed as part of a transition (状态转换过程中执行的活动)

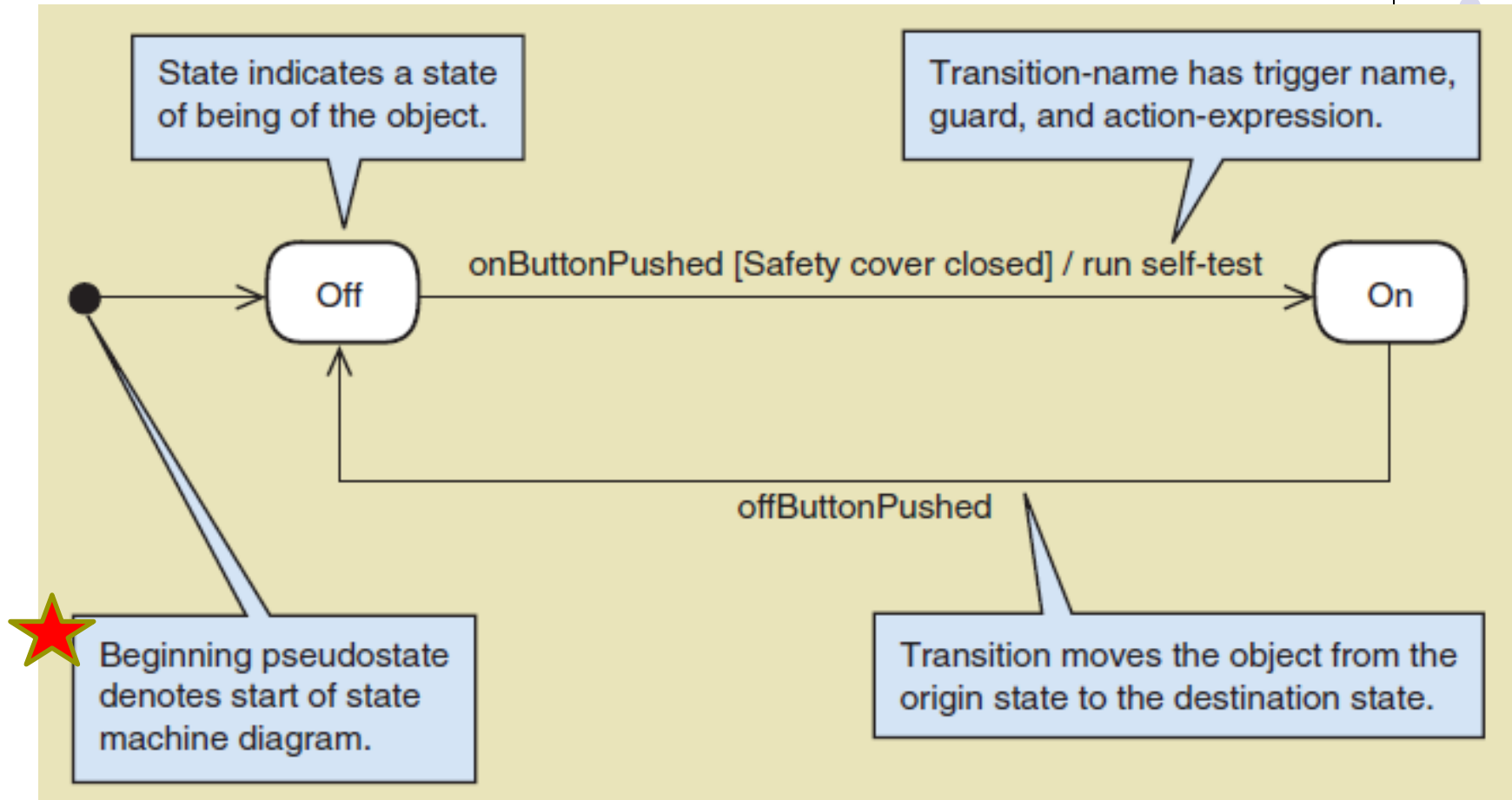
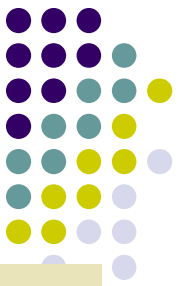
State Machine Diagram (continued)



- Pseudo state (伪状态)
 - The starting point of a state machine diagram (black dot) (状态机图的起始点)
- Origin state (原始状态)
 - The original state of an object before transition (对象在转换之前的原始状态)
- Destination state (目标状态)
 - The state to which the object moves after the transition (对象在转换后移动到的状态)
- Guard condition (保护条件)
 - A true false test to see whether a transition can fire (一个真假测试，看一个转换是否可以触发)

State Machine Diagram

for a Printer

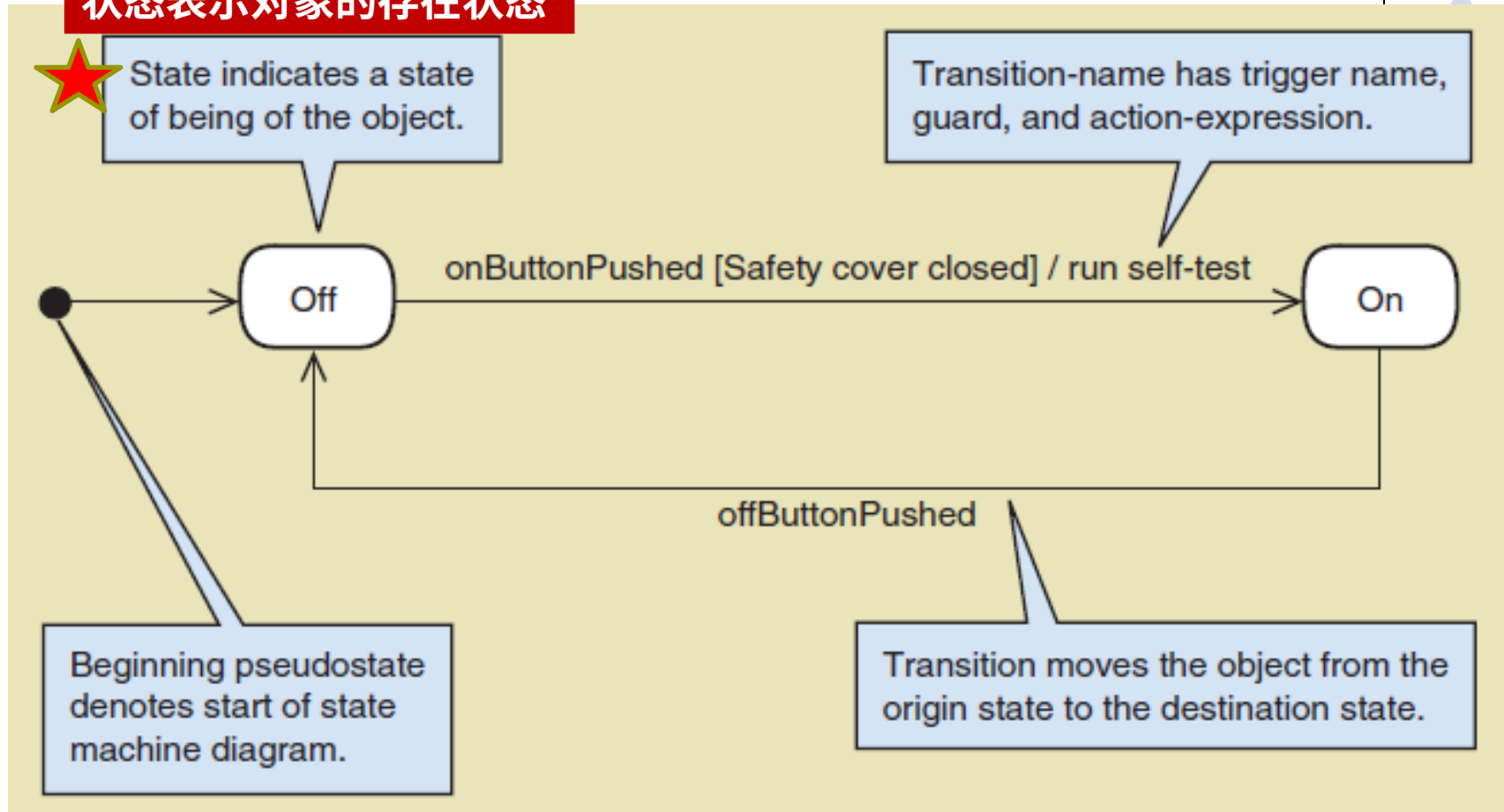


开始伪状态：表示状态活动图的开始

State Machine Diagram

for a Printer

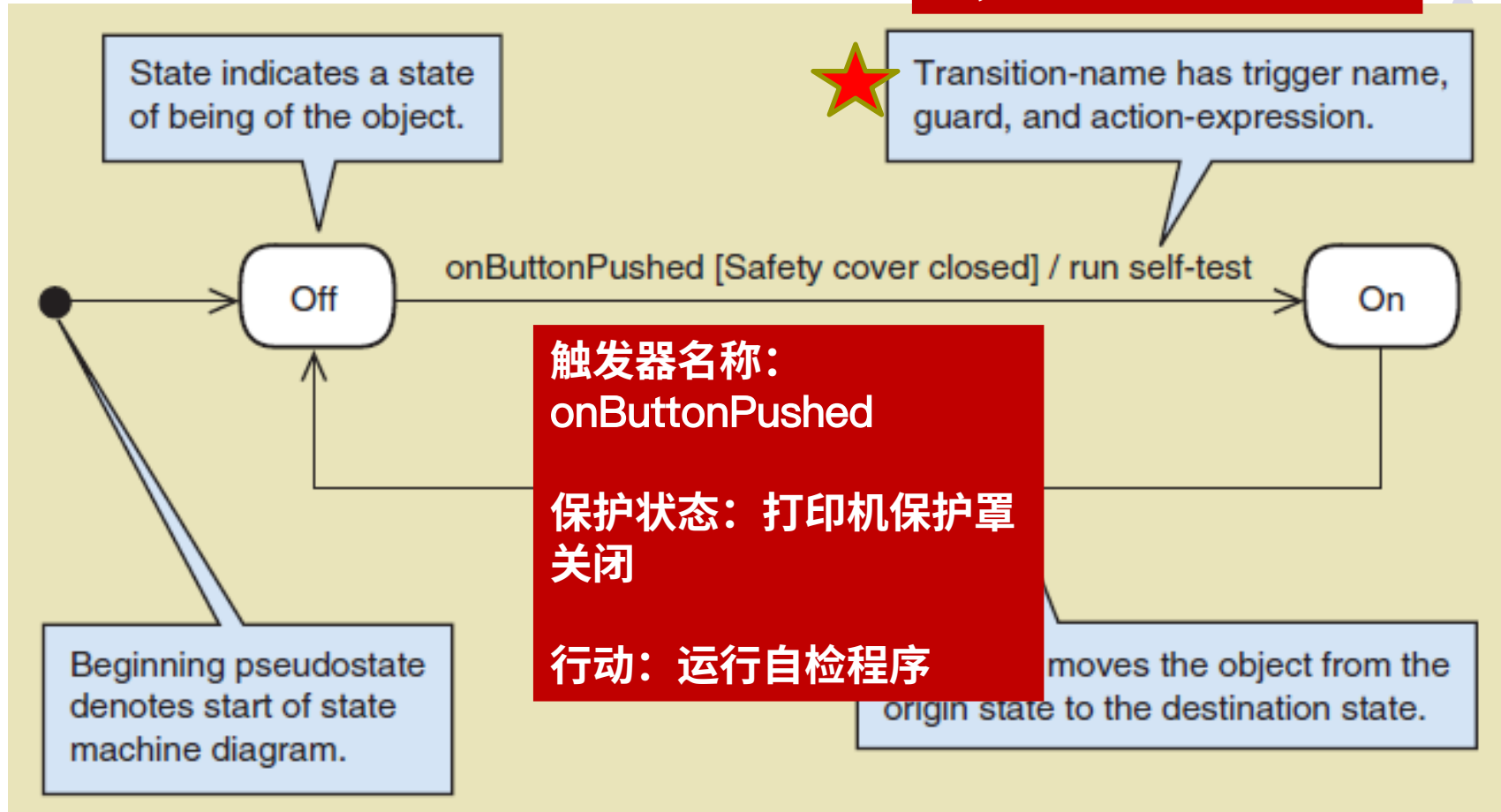
状态表示对象的存在状态



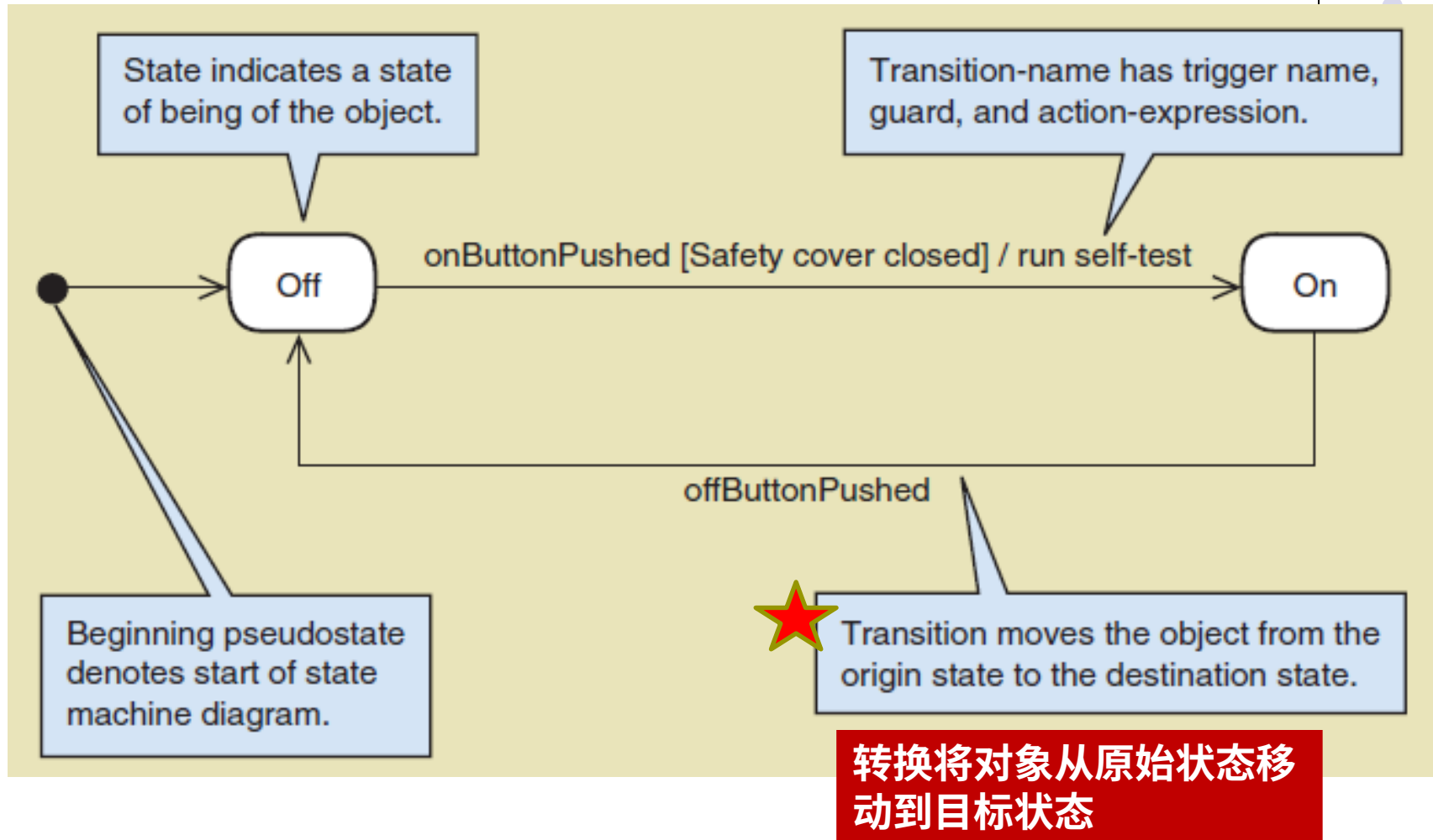
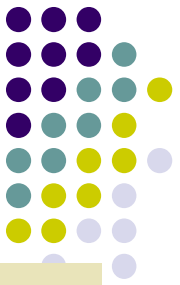
State Machine Diagram

for a Printer

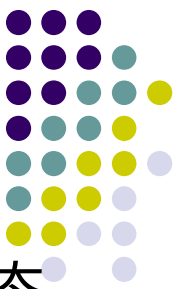
转换名包括触发器的名称，保护状态和行动表达



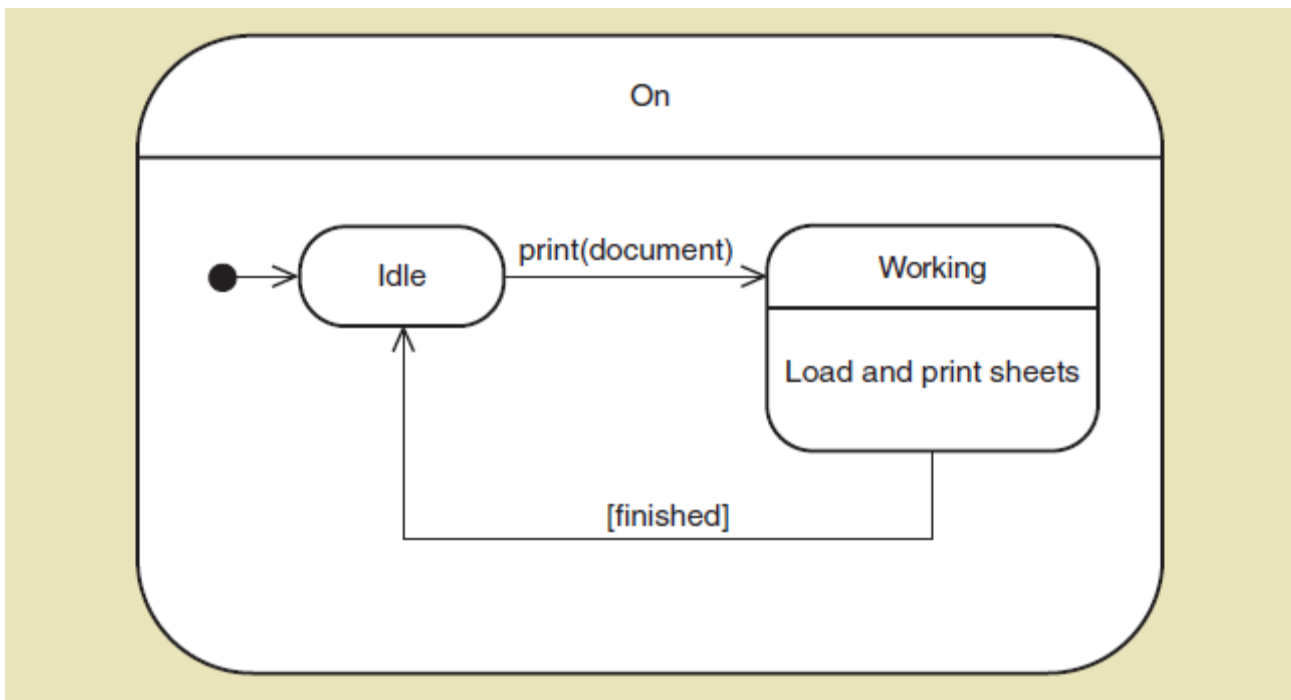
State Machine Diagram for a Printer



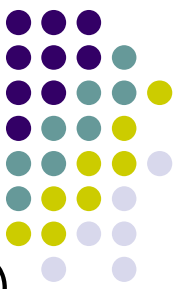
Composite States (复合状态)



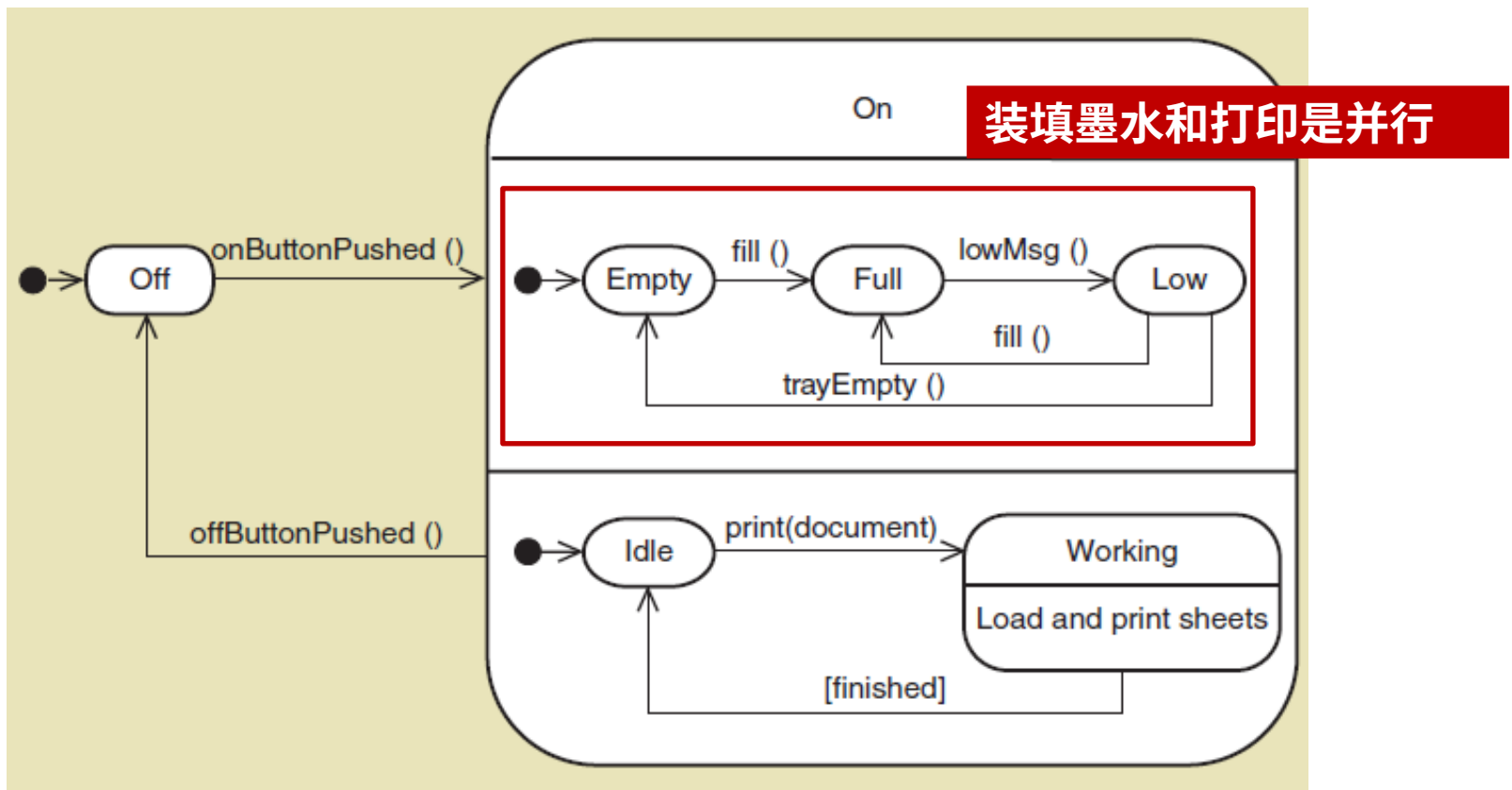
- State containing other states and transitions (包含其他状态和转换的状态)
- Printer can be On and either Idle or Working (打印机可以打开，空闲或工作)



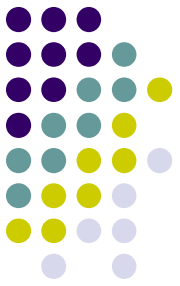
Concurrent Paths (并行路径)



- Multiple paths in composite state (多个路径处于复合状态)
- Printer On paths are independent (打印机路径是独立的)

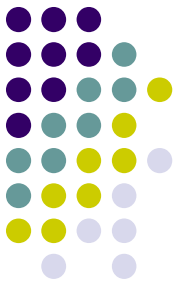


Steps for Developing State Machine Diagram



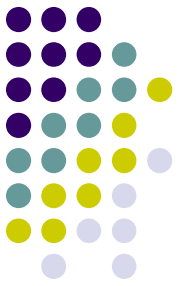
1. Review the class diagram and **select classes** that might require state machine diagrams （回顾类图并选出需要状态机图的类）
2. For each class, make a list of **status conditions** (states) you can identify （对于组中每一个选出的类，列出你能确定的所有状态）
3. Begin building diagram fragments by **identifying transitions** that cause an object to leave the identified state （通过确定导致对象离开特定状态的转换来开始建立状态机图片段）

Steps for Developing State Machine Diagram



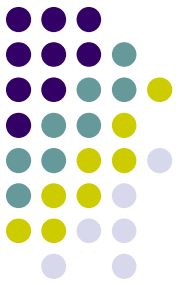
4. Sequence these states in the **correct order** and aggregate combinations into larger fragments
(按正确顺序将这些状态 - 转换组合起来)
4. **Review paths** and look for independent, concurrent paths (回顾这些路径并找出独立且并发的路径)
6. Look for **additional transitions** and test both directions (查找其他的转换)
6. Expand each transition with appropriate message event, guard condition, and action expression
(用合适的消息事件、判定条件和行为描述扩展每个转换)

Steps for Developing State Machine Diagram (continued)

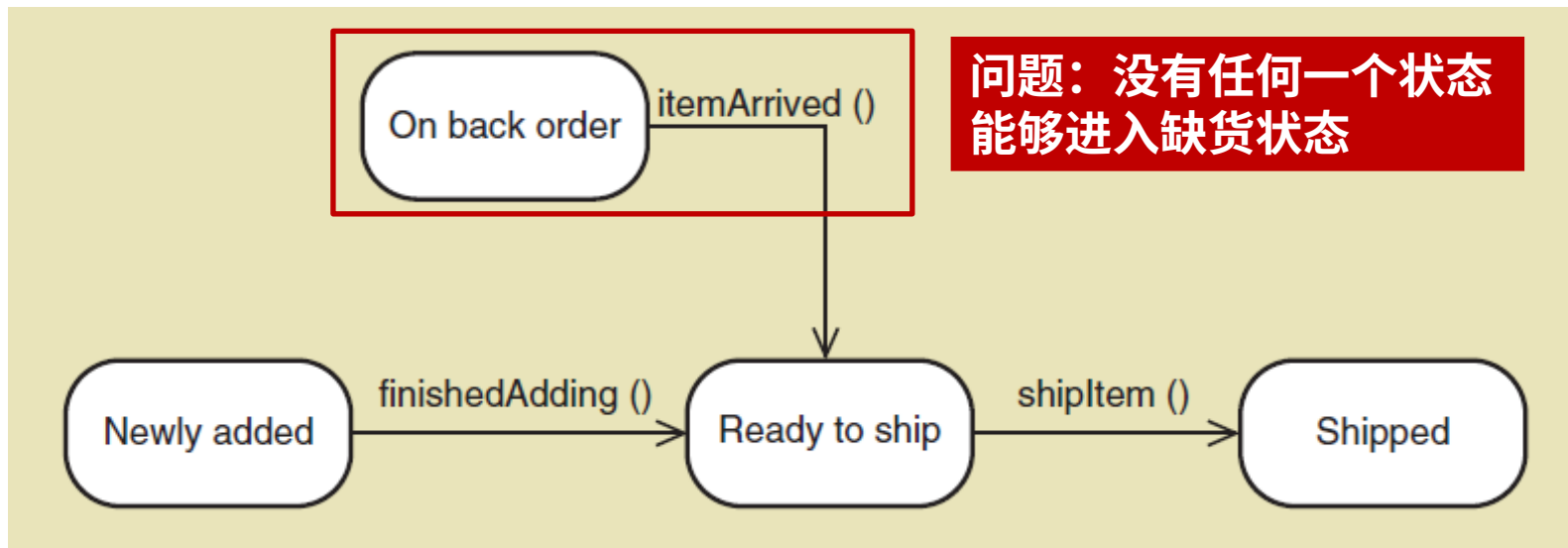


6. Review and test the state machine diagram for the class (回顾并检查状态机图)
 - Make sure **state are really state** for the object in the class (确保状态确实是类对象的状态)
 - **Follow the life cycle** of an object coming into existence and being deleted (从产生到被系统删除, 跟踪对象的生命周期)
 - Be sure the diagram covers all **exception condition** (确保状态机图覆盖了所有异常情况以及正常的行为流)
 - Look again for **concurrent paths** and **composite states** (再次检查并发行为 (多路径) 和可能得嵌套路径 (复合状态))

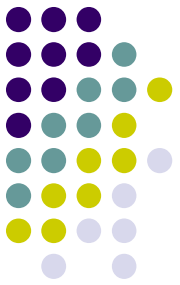
RMO Domain Class States for SaleItem Object



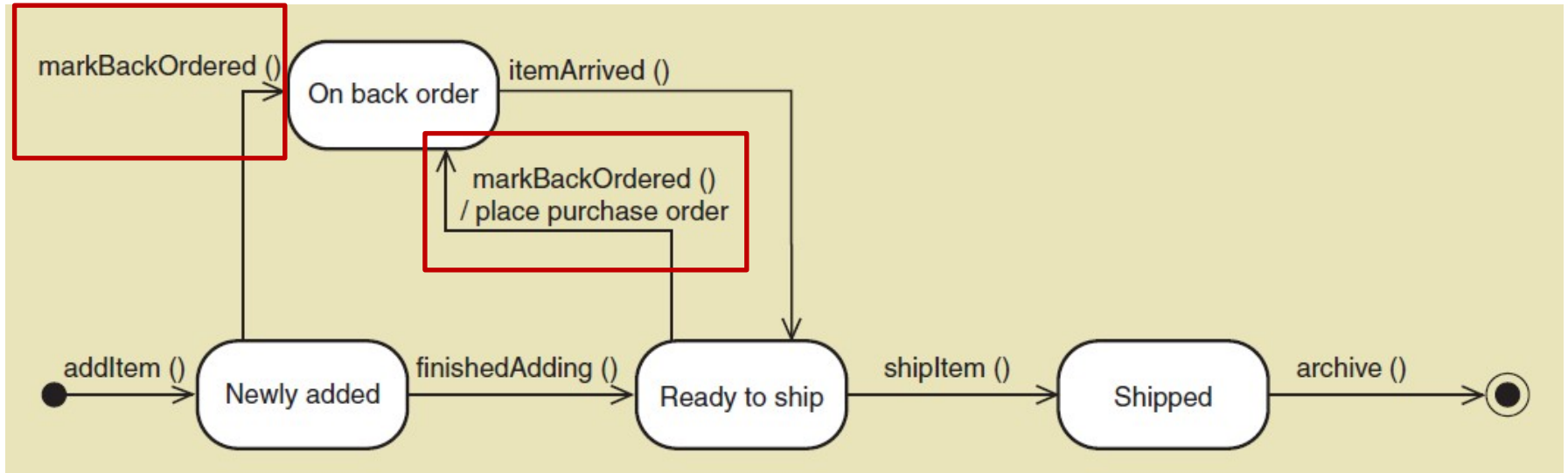
State	Transition causing exit from state
<i>Newly added</i>	finishedAdding
<i>Ready to ship</i>	shipItem
<i>On back order</i>	itemArrived
<i>Shipped</i>	No exit transition defined



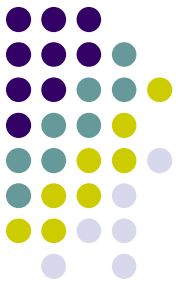
Final State Machine Diagram for SaleItem Object



- addItem() and archive() transitions added
- markBackOrdered() transition added

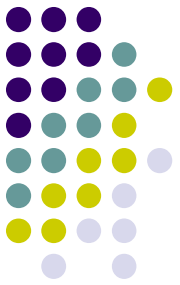


RMO Domain Class States for Sale Object

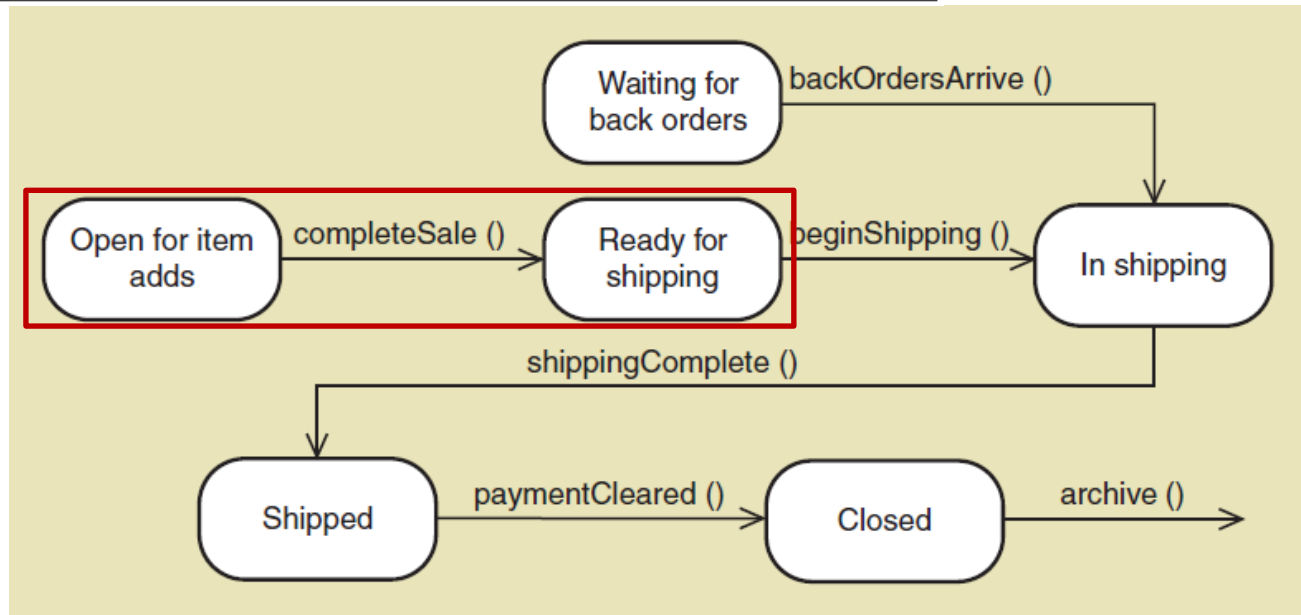


State	状态	Exit transition	转移
<i>Open for item adds</i>	增加物品	completeSale	完成订购
<i>Ready for shipping</i>	准备运输	beginShipping	开始运输
<i>In shipping</i>	正在运输	shippingComplete	运输完成
<i>Waiting for back orders</i>	等待缺货物品	backOrdersArrive	缺货物品到货
<i>Shipped</i>	已到货	paymentCleared	支付证明
<i>Closed</i>	结束	archive	存档

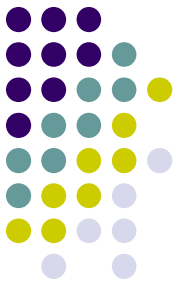
Initial State Machine Diagram for RMO Sale Object



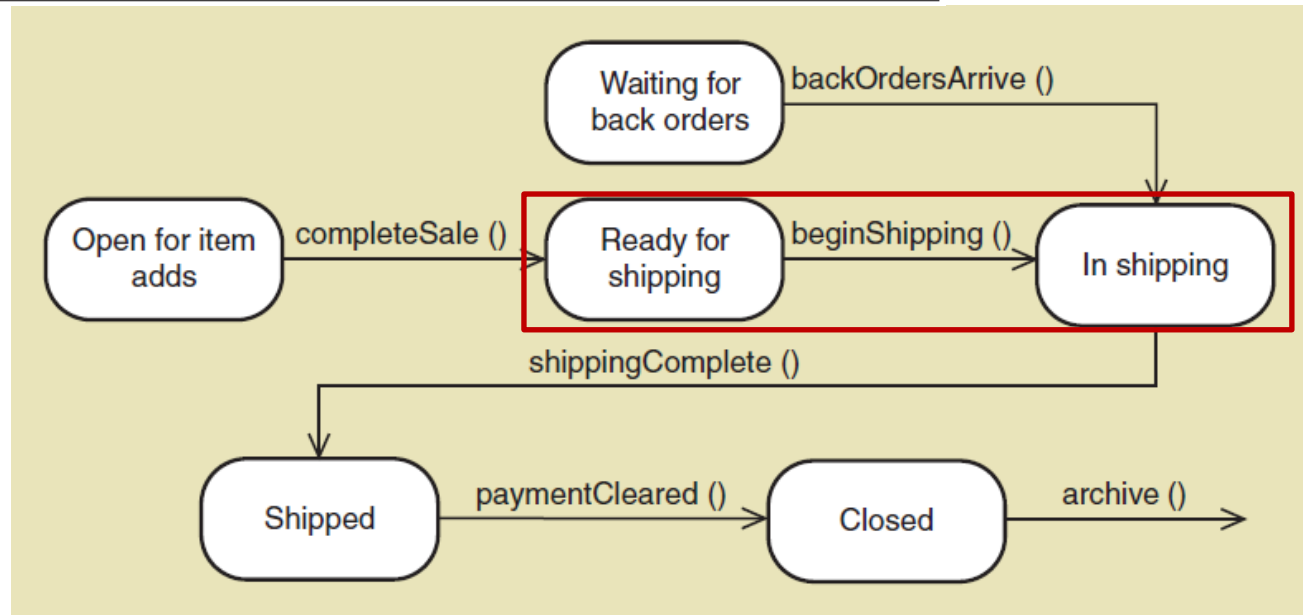
State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive



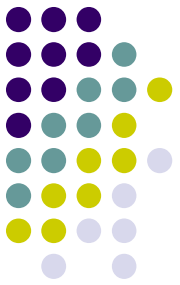
Initial State Machine Diagram for RMO Sale Object



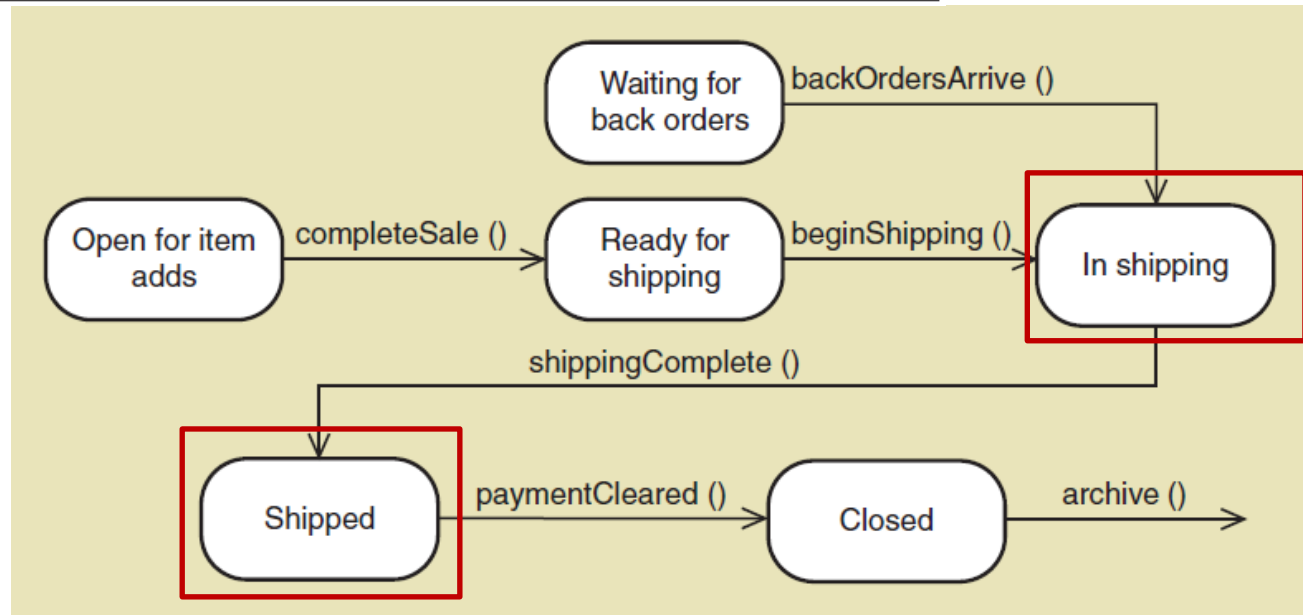
State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive



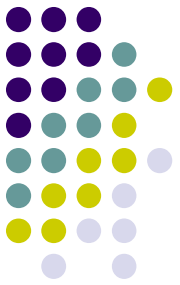
Initial State Machine Diagram for RMO Sale Object



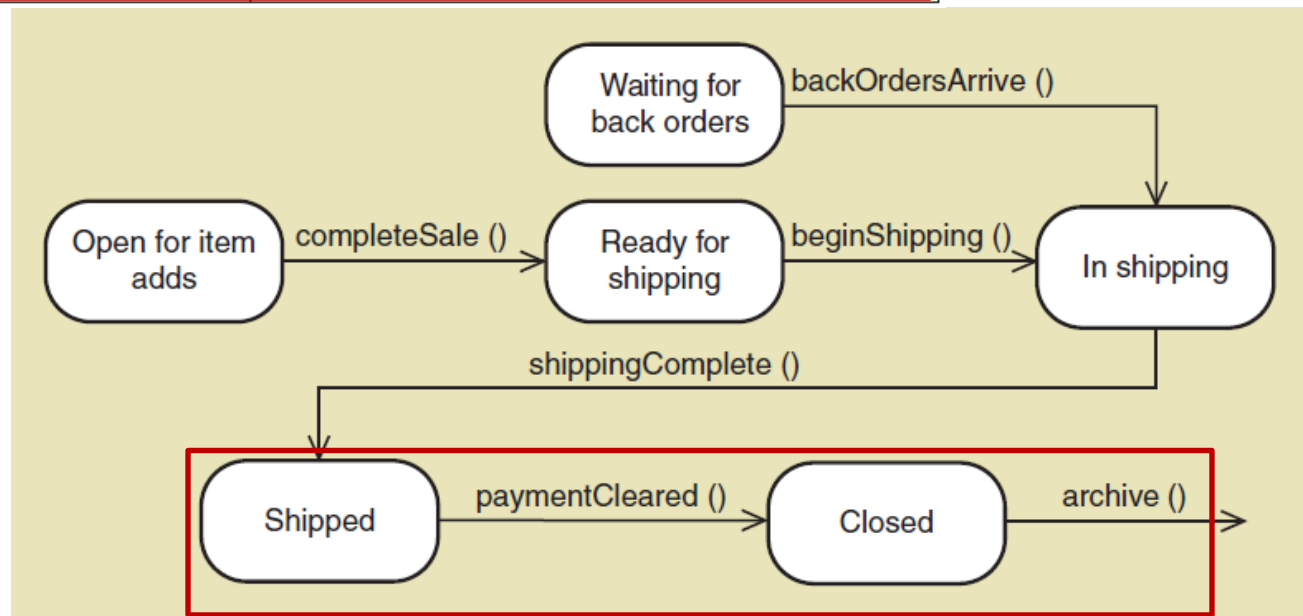
State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive



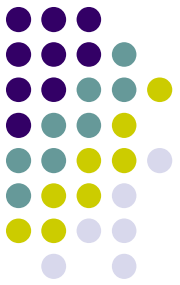
Initial State Machine Diagram for RMO Sale Object



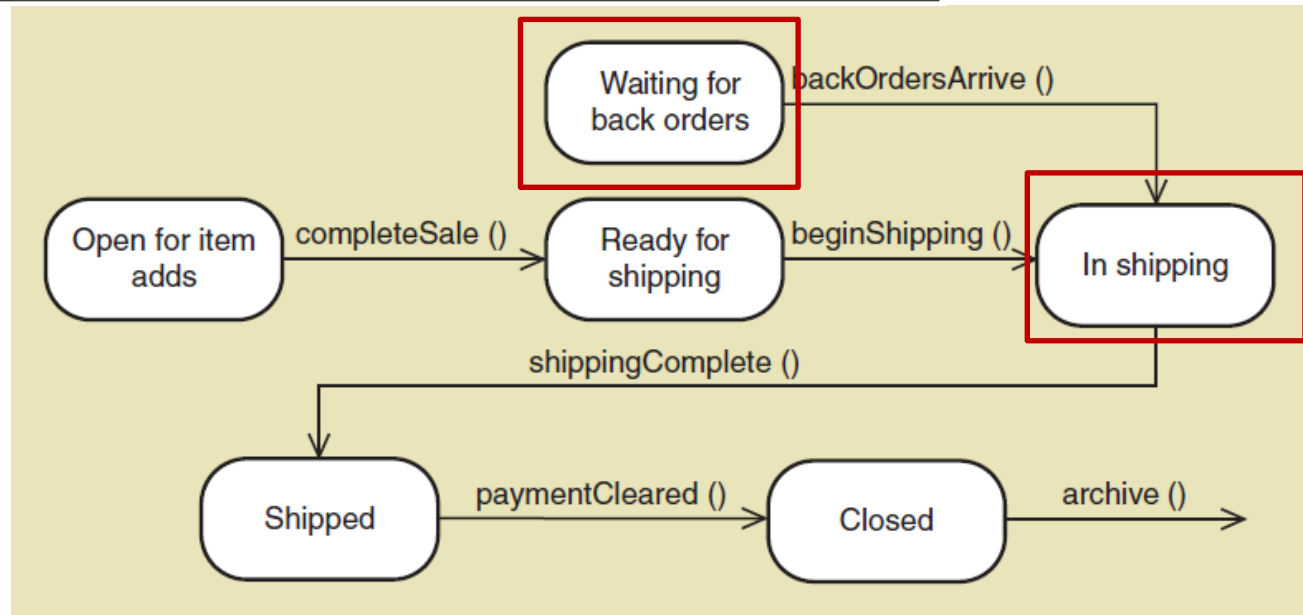
State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive



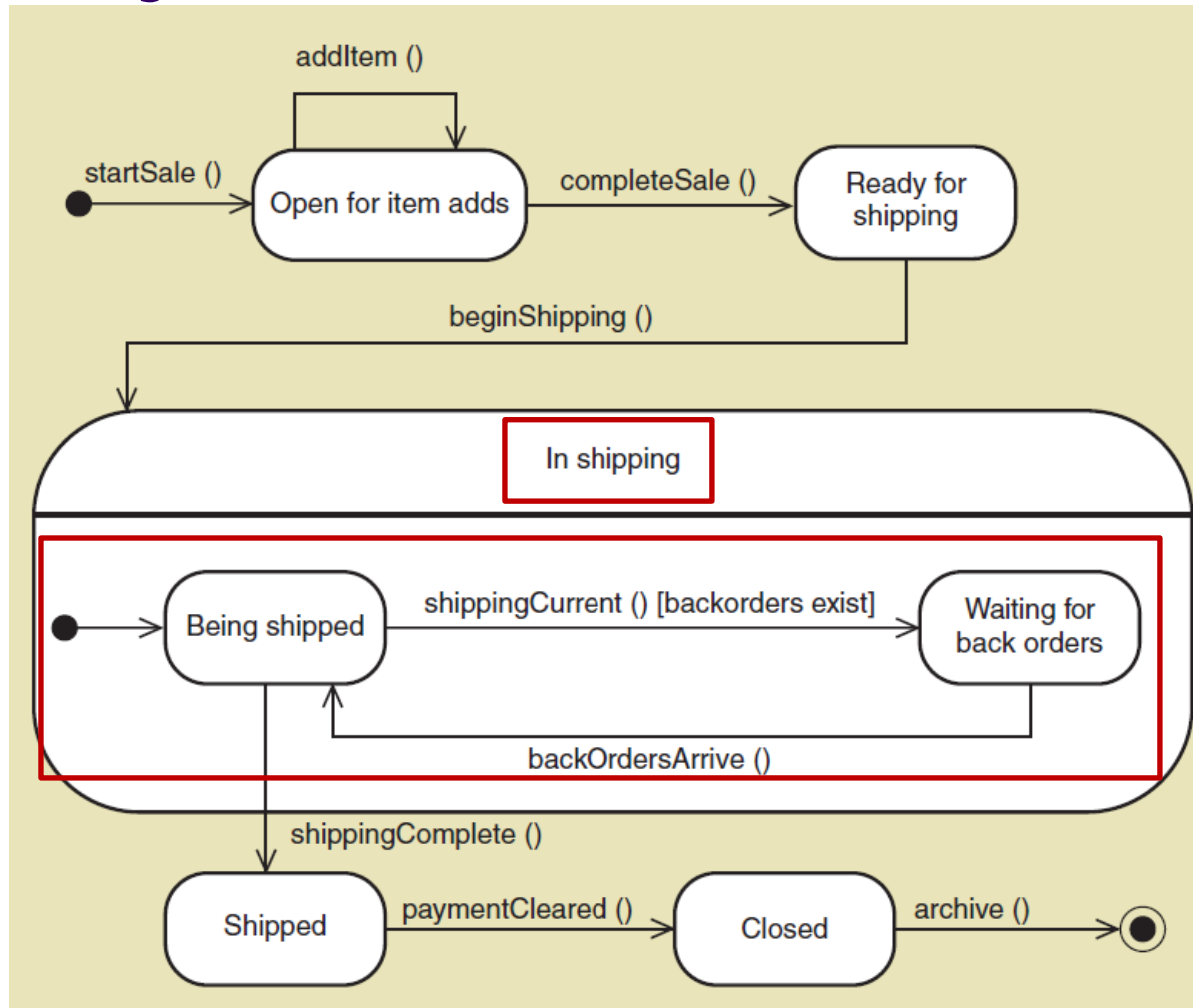
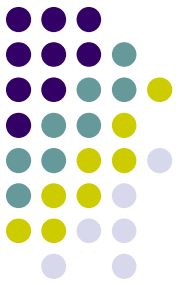
Initial State Machine Diagram for RMO Sale Object

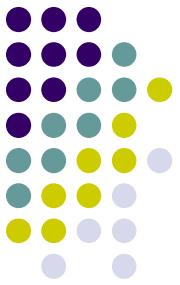


State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive



Final State Machine Diagram for Sale Object

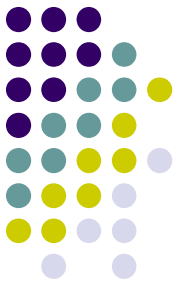




Chapter 5 Outline

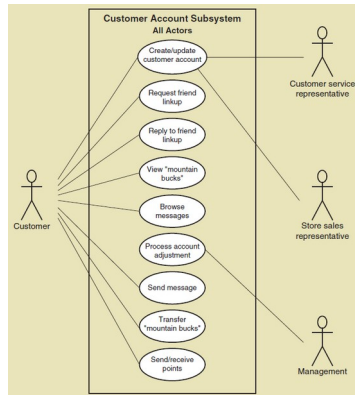
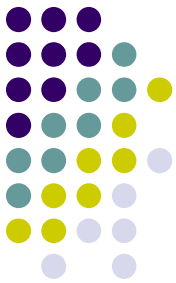
- Use Case Descriptions
- Activity Diagrams for Use Cases
- The System Sequence Diagram—Identifying Inputs and Outputs
- The State Machine Diagram—Identifying Object Behavior
- Integrating Requirements Models (集成需求模型)

Extending and Integrating Requirements Models



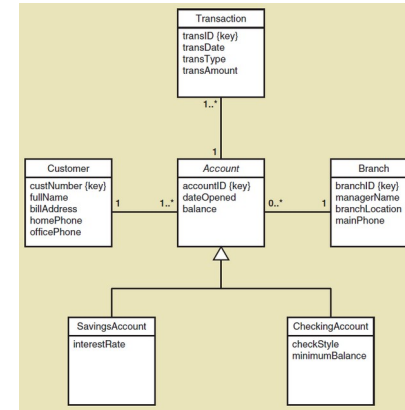
- Use cases （用例）
 - Use case diagram 用例图 （ Sec 3.6 ）
 - Use case description 用例描述 （ Sec 5.2 ）
 - Activity diagram 用例活动图 （ Sec 5.3 ）
 - System sequence diagram (SSD) 系统顺序图 （ Sec 5.4 ）
- Domain Classes （域类）
 - Domain model class diagram 域模型类图 (Sec 4.4)
 - State machine diagram 状态机图 (Sec 5.5)

Integrating Requirements Models



Sec 3.6

Use case diagrams



Sec 4.4

Domain model class diagram

Use case descriptions

Activity diagrams

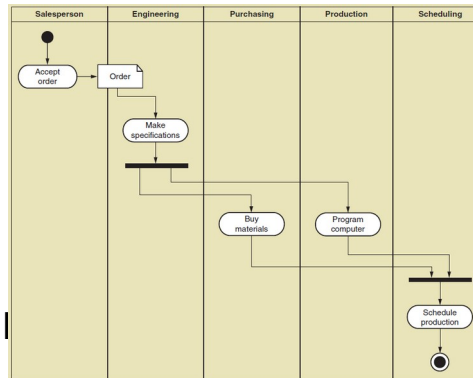
System sequence diagrams (SSDs)

State machine diagrams

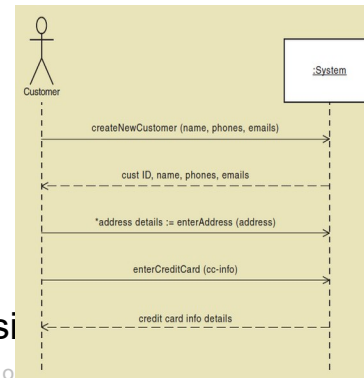
Sec 5.2

Use case name:	Create customer account.								
Scenario:	Create online customer account.								
Triggering event:	New customer wants to set up account online.								
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.								
Actors:	Customer.								
Related use cases:	Customer account subsystem must be available. Check out shopping cart use case.								
Stakeholders:	Accounting, Marketing, Sales.								
Preconditions:	Customer account subsystem must be available. Credit/debit authorization services must be available.								
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.								
Flow of activities:	<table border="1"> <thead> <tr> <th>Actor</th><th>System</th></tr> </thead> <tbody> <tr> <td>1. Customer indicates desire to create customer account and enters basic customer information.</td><td>1.1 System creates a new customer.</td></tr> <tr> <td>2. Customer enters one or more addresses.</td><td>2.1 System creates addresses. 2.2 System prompts for credit/debit card.</td></tr> <tr> <td>3. Customer enters credit/debit card information.</td><td>3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.</td></tr> </tbody> </table>	Actor	System	1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer.	2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.	3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.
Actor	System								
1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer.								
2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.								
3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.								
Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.								

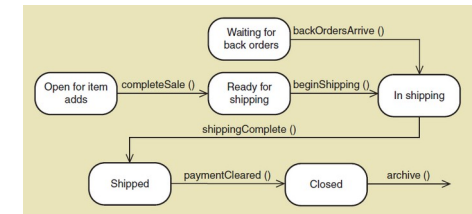
Sec 5.3



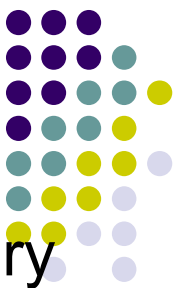
Sec 5.4



Sec 5.5

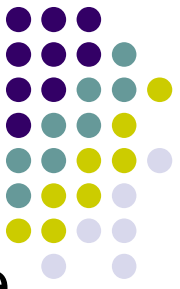


Summary



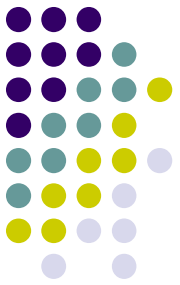
- Chapters 3 and 4 identified and modeled the two primary aspects of functional requirements: *use cases* and *domain classes* (第3章和第4章确定并建模了功能需求的两个主要方面：用例和域类)
 - This chapter focuses on additional techniques and models to extend the requirements models to show more detail (本章着重于额外的技术和模型来扩展需求模型以显示更多的细节)
- Fully developed *use case descriptions* provide information about each use case, including actors, stakeholders, preconditions, post conditions, the flow of activities and exceptions conditions (完全开发的用例描述提供了关于每个用例的信息，包括参与者、相关人员、前提条件、后置条件、活动流和异常状态)

Summary



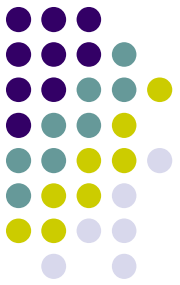
- *Activity diagrams* (first shown in Chapter 2) can also be used to show the flow of activities for a use case (活动图 (第一次在第 2 章中展示) 也可以用来显示一个用例的活动流)
- *System sequence diagrams* (SSDs) show the inputs and outputs for each use case as messages (系统顺序图 (ssd) 将每个用例的输入和输出显示为消息)
- *State machine diagrams* show the states an object can be in over time between use cases (状态机图显示了一个对象在用例之间随时间推移可能处于的状态)

Summary (continued)



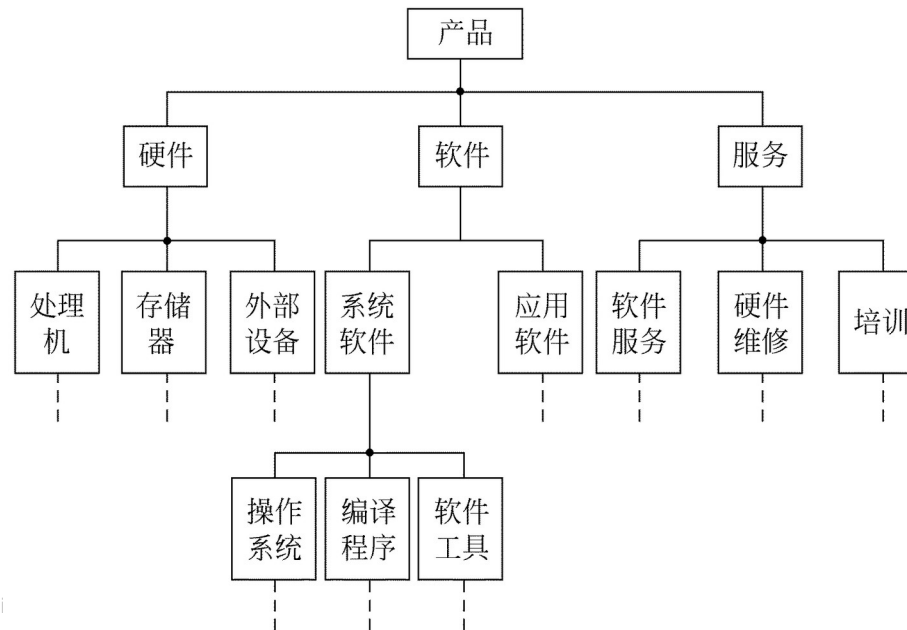
- Use cases are modeled in more detail using fully developed use case descriptions, activity diagrams, and system sequence diagrams (使用完全开发的用例描述、活动图和系统顺序图对用例进行更详细的建模)
- Domain classes are modeled in more detail using state machine diagrams (使用状态机图对域类进行更详细的建模)
- Not all use cases and domain classes are modeled at this level of detail. Only model when there is complexity and a need to communicate details (并不是所有的用例和领域类都在这个细节级别上建模。只有在存在复杂性和需要沟通细节时才建模)

补充 1：其他图形工具

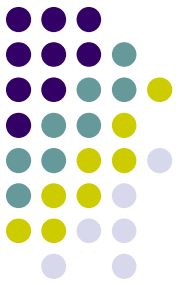


● 1. 层次方框图

- 层次方框图用树形结构的一系列多层次的矩形框描绘数据的层次结构。
- 例如，描绘一家计算机公司全部产品的数据结构可以用下图的层次方框图表示。这家公司的产品由硬件、软件和服务 3 类产品组成。



补充 1：其他图形工具



● 2. IPO 图

- IPO 图是输入加工输出（Input Process Output）图的简称
- 在系统的模块结构图形成过程中，产生了大量的模块，在进行详细设计时开发者应为每一个模块写一份说明。
IPO 图就是用来说明每个模块的输入、输出数据和数据加工的重要工具。

补充 1：其他图形工具



● 2. IPO 图

IPO 表

系统：_____ 作者：_____

模块：_____ 日期：_____

编号：_____

被调用：

调用：

输入：

输出：

处理：

局部数据元素： 注释：

IPO 表

系统： 旅行社机票预订系统 作者： 程序鸡

模块： 输入预定机票信息 日期： 2020/12/16

编号： 001

被调用： 安排航班模块 调用： 预定机票信息

输入： 预定机票信息 输出： 预定机票信息记录

处理： 将输入的预定机票信息封装之后保存在一个记录中

局部数据元素： 预定机票信息 注释：

IPO 表

系统： 旅行社机票预订系统 作者： 程序鸡

模块： 安排航班 日期： 2020/12/16

编号： 002

被调用： 安排航班模块 调用： 航班安排信息

输入： 预定机票信息 输出： 航班安排信息记录

处理： 将预定机票信息进行分析，匹配合适的航班，生成航班安排

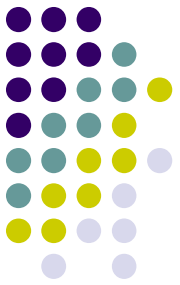
局部数据元素： 预定机票信息、航班信息 注释：

补充 2：验证软件需求



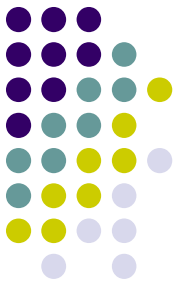
- 从哪些方面验证软件需求
- 需求分析阶段的工作结果是开发软件系统的重要基础，大量统计数字表明，**软件系统中 15% 的错误起源于错误的需求**
- **验证软件需求正确性的 4 个方面**
 - **一致性**：当需求分析的结果是用自然语言书写的时候，除了靠人工技术审查验证软件系统规格说明书的正确性之外，目前还没有其他更好的“测试”方法。
 - **完整性**：只有目标系统的用户才真正知道软件需求规格说明书是否完整、准确地描述了他们的需求。
 - **现实性**：为了验证需求的现实性，分析员应该参照以往开发类似系统的经验，分析用现有的软、硬件技术实现目标系统的可能性。
 - **有效性**：许多用户并不能清楚地认识到他们的需要，不能有效地比较陈述需求的语句和实际需要的功能。

补充 3：形式化语言



- 非形式化方法的缺点
- 用自然语言书写的系统规格说明书，可能存在矛盾、二义性、含糊性、不完整性及抽象层次混乱等问题。
 - 矛盾是指一组相互冲突的陈述。
 - 二义性是指读者可以用不同方式理解的陈述。
 - 含糊性
 - 不完整性
 - 抽象层次混乱是指在非常抽象的陈述中混进了一些关于细节的低层次陈述。

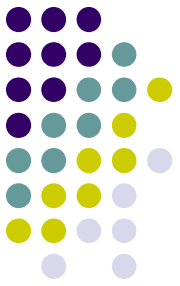
补充 3：形式化语言



- 形式化方法的优点
- 为了克服非形式化方法的缺点，人们把数学引入软件开发过程，创造了基于数学的形式化方法。
 - 数学能够简洁准确地描述物理现象、对象或动作的结果，因此是理想的建模工具。
 - 数学以在不同的软件工程活动之间平滑地过渡。
 - 数学提供了高层确认的手段。

补充 3：形式化语言

- 形式化方法
 - 有穷状态机
 - Petri 网

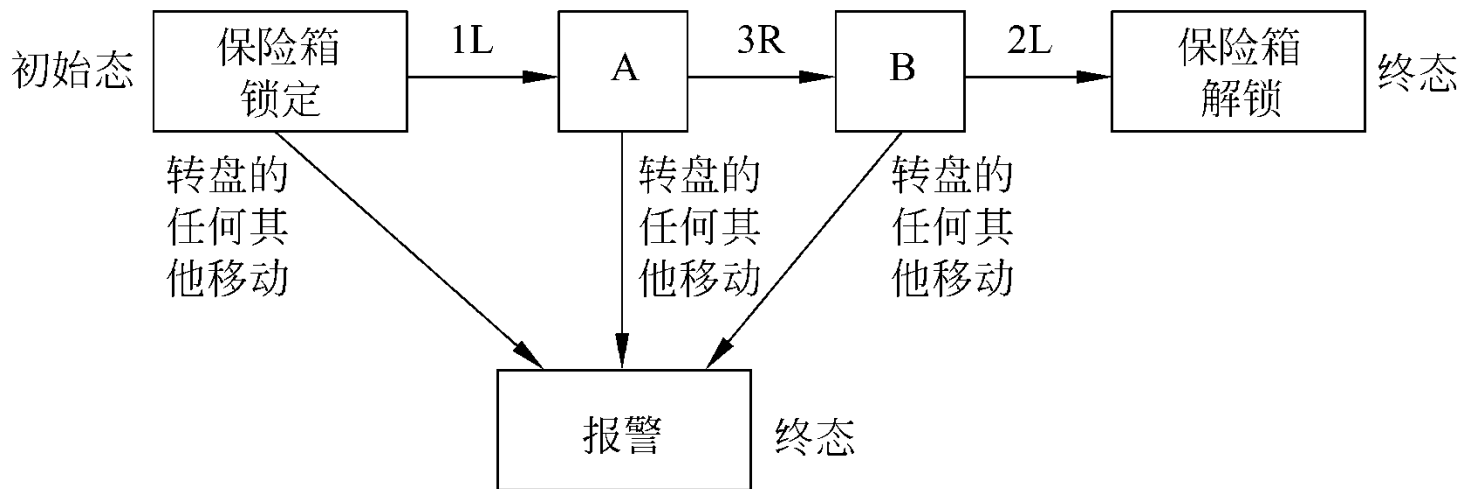


补充 3：形式化语言

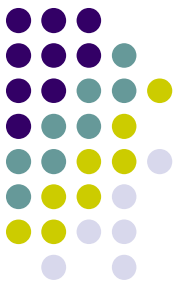


● 形式化方法 1：有穷状态机

- 例子：一个保险箱上装了一个复合锁，锁有 3 个位置，分别标记为 1、2、3，转盘可向左 (L) 或向右 (R) 转动。这样，在任意时刻转盘都有 6 种可能的运动，即 1L、1R、2L、2R、3L 和 3R。保险箱的组合密码是 1L、3R、2L，转盘的任何其他运动都将引起报警。



补充 3：形式化语言



- 形式化方法 1：有穷状态机
 - 一个初始态：保险箱锁定状态。
 - 下表为保险箱的状态转换表

当前状态 次态 转盘动作	保险箱锁定	A	B
1L	A	报警	报警
1R	报警	报警	报警
2L	报警	报警	保险箱解锁
2R	报警	报警	报警
3L	报警	报警	报警
3R	报警	B	报警

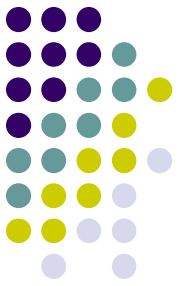
补充 3：形式化语言



● 形式化方法 1：有穷状态机

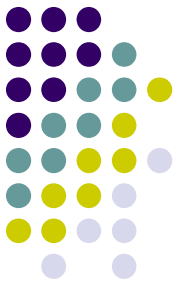
- 从上面这个简单的例子可以看出，一个有穷状态机包括下述 5 个部分：
 - 状态集 J ：{保险箱锁定，A，B，保险箱解锁，报警}。
 - 输入集 K ：{1L，1R，2L，2R，3L，3R}。
 - 转换函数 T ：如表所示。
 - 初始态 S ：保险箱锁定。
 - 终态集 F ：{保险箱解锁，报警}。

补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 如果使用更形式化的术语，一个有穷状态机可以表示为一个 5 元组 (J, K, T, S, F) ，其中：
 - J 是一个有穷的非空状态集；
 - K 是一个有穷的非空输入集；
 - T 是一个从 $(J-F) \times K$ 到 J 的转换函数；
 - $S \in J$ ，是一个初始状态；
 - $F \in J$ ，是终态集。

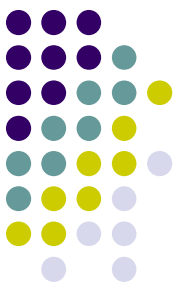
补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 有穷状态机的概念在计算机系统中应用得非常广泛。
- 例如，每个菜单驱动的用户界面都是一个有穷状态机的实现。一个菜单的显示和一个状态相对应，键盘输入或用鼠标选择一个图标是使系统进入其他状态的一个事件。状态的每个转换都具有下面的形式：

当前状态〔菜单〕 + 事件〔所选择的项〕 => 下个状态

补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 为了对一个系统进行规格说明，通常都需要对有穷状态机做一个很有用的扩展，即在前述的 5 元组中加入第 6 个组件——谓词集 P ，从而把有穷状态机扩展为一个 6 元组，其中每个谓词都是系统全局状态 Y 的函数。转换函数 T 现在是一个从 $(J-F) \times K \times P$ 到 J 的函数。现在的转换规则形式如下：

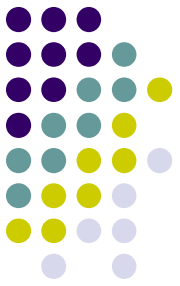
当前状态〔菜单〕 + 事件〔所选择的项〕 + 谓词 $\Rightarrow$ 下个状态

补充 3：形式化语言



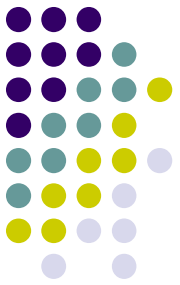
- 形式化方法 1：有穷状态机
- 例子：用自然语言描述的对电梯系统的需求。
- 在一幢 m 层的大厦中需要一套控制 n 部电梯的产品，要求这 n 部电梯按照约束条件 $C1$ ， $C2$ 和 $C3$ 在楼层间移动。
 - **C1**：每部电梯内有 m 个按钮，每个按钮代表一个楼层。当按下一个按钮时该按钮指示灯亮，同时电梯驶向相应的楼层，到达按钮指定的楼层时指示灯熄灭。
 - **C2**：除了大厦的最低层和最高层之外，每层楼都有两个按钮分别请求电梯上行和下行。这两个按钮之一被按下时相应的指示灯亮，当电梯到达此楼层时灯熄灭，电梯向要求的方向移动。
 - **C3**：当对电梯没有请求时，它关门并停在当前楼层。

补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 使用一个扩展的有穷状态机对本产品进行规格说明。
- 分析：问题中有两个按钮集。n 部电梯中的每一部都有 m 个按钮，一个按钮对应一个楼层。因为这 $m \times n$ 个按钮都在电梯中，为**电梯按钮**。每层楼有两个按钮，一个请求向上，另一个请求向下，称为**楼层按钮**。

补充 3：形式化语言



- 形式化方法 1：有穷状态机

- 电梯按钮的状态转换图如下图所示：

- 令 $EB(e,f)$ 表示按下电梯 e 内的按钮并请求到 f 层去。 $EB(e,f)$ 有两个状态，分别是按钮发光 (打开) 和不发光 (关闭)。状态是：

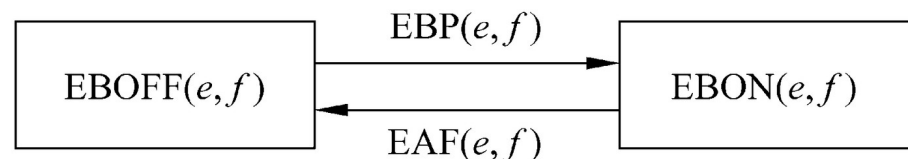
$EBON(e,f)$ ：电梯按钮 (e,f) 打开

$EBOFF(e,f)$ ：电梯按钮 (e,f) 关闭

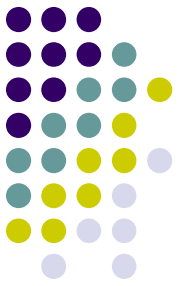
- 如果电梯按钮 (e,f) 发光且电梯到达 f 层，该按钮将熄灭。相反如果按钮熄灭，则按下它时，按钮将发光。两个事件，它们分别是：

$EBP(e,f)$ ：电梯按钮 (e,f) 被按下

$EAF(e,f)$ ：电梯 e 到达 f 层



补充 3：形式化语言

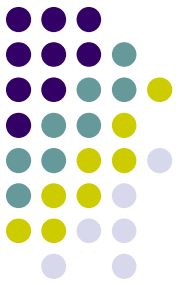


- 形式化方法 1：有穷状态机
- 为了定义与这些事件和状态相联系的状态转换规则，
- 需要一个谓词 $V(e,f)$ ，它的含义如下：

$V(e,f)$ ：电梯 e 停在 f 层

- 如果电梯按钮 (e,f) 处于关闭状态〔当前状态〕，而且电梯按钮 (e,f) 被按下〔事件〕，而且电梯 e 不在 f 层〔谓词〕，则该电梯按钮打开发光〔下个状态〕。

补充 3：形式化语言

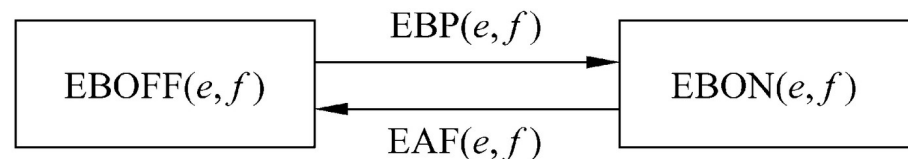


- 形式化方法 1：有穷状态机
- 状态转换规则的形式化描述如下：

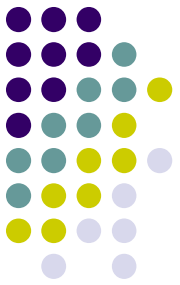
$$\text{EBOFF}(e,f) + \text{EBP}(e,f) + \text{not } V(e,f) \Rightarrow \text{EBON}(e,f)$$

- 反之，如果电梯到达 f 层，而且电梯按钮是打开的，于是它就会熄灭。
- 这条转换规则可以形式化地表示为：

$$\text{EBON}(e,f) + \text{EAF}(e,f) \Rightarrow \text{EBOFF}(e,f)$$



补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 楼层按钮的有穷状态机：令 $FB(d,f)$ 表示 f 层请求电梯向 d 方向运动的按钮，楼层按钮 $FB(d,f)$ 的状态转换图如图所示，楼层按钮的状态如下。

$FBON(d,f)$ ：楼层按钮 (d,f) 打开

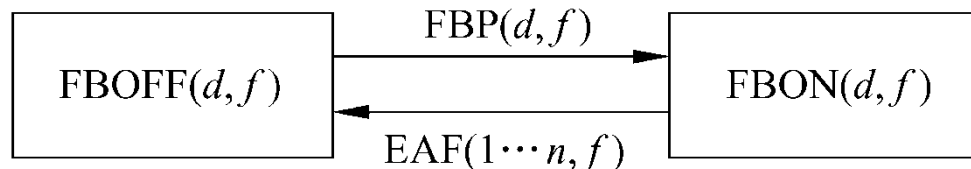
$FBOFF(d,f)$ ：楼层按钮 (d,f) 关闭

- 楼层按钮已经打开，一部电梯到达 f 层则按钮关闭。
- 楼层按钮原来是关闭的，被按下后该按钮将打开。

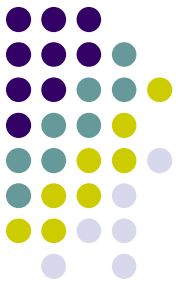
$FBP(d,f)$ ：楼层按钮 (d,f) 被按下

$EAF(1\dots n,f)$ ：电梯 1 或...或 n 到达 f 层

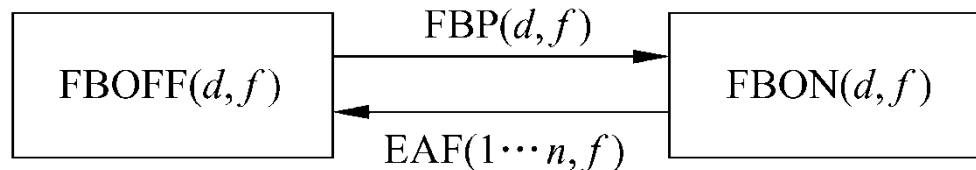
- 其中， $1\dots n$ 表示或为 1 或为 2... 或为 n 。



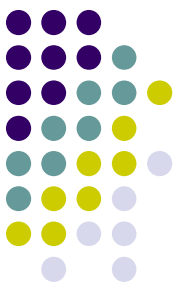
补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 为了定义与这些事件和状态相联系的状态转换规则，需要一个谓词 $S(d,e,f)$ ，定义如下： $S(d,e,f)$ ：电梯 e 停在 f 层并且移动方向由 d 确定为向上 ($d=U$) 或向下 ($d=D$)。
- 这个谓词是一个状态，形式化方法允许把事件和状态作为谓词对待。
- 使用谓词 $S(d,e,f)$ ，形式化转换规则为：
 $\text{FBOFF}(d,f) + \text{FBP}(d,f) + \text{not } S(d,1\dots n,f) \Rightarrow \text{FBON}(d,f)$
 $\text{FBON}(d,f) + \text{EAF}(1\dots n,f) + S(d,1\dots n,f) \Rightarrow \text{FBOFF}(d,f)$
- 其中， $d=U \text{ or } D$ 。



补充 3：形式化语言



- 形式化方法 1：有穷状态机
- 有穷状态机方法采用了一种简单的格式来描述规格说明：

当前状态 + 事件 + 谓词 => 下个状态

- 优点：这种形式的规格说明易于书写、易于验证，而且可以比较容易地把它转变成设计或程序代码。有穷状态机方法比数据流图技术更精确，而且和它一样易于理解。
- 缺点：在开发一个大系统时三元组（即状态、事件、谓词）的数量会迅速增长。此外，和数据流图方法一样，形式化的有穷状态机方法也没有处理定时需求。

补充 3：形式化语言



- 形式化方法 2：Petri 网

- Petri 网由来：

- 并发系统中遇到的一个主要问题是定时问题。这个问题可以表现为多种形式，如同步问题、竞争条件以及死锁问题。
- Petri 网是由 Carl Adam Petri 发明的。在性能评价、操作系统和软件工程等领域，Petri 网应用得都比较广泛。特别是已经证明，用 Petri 网可以有效地描述并发活动。

- Petri 网用户：

- 用于确定系统中隐含的定时问题的一种有效技术是 Petri 网，这种技术的一个很大的优点是它也可以用于设计中。

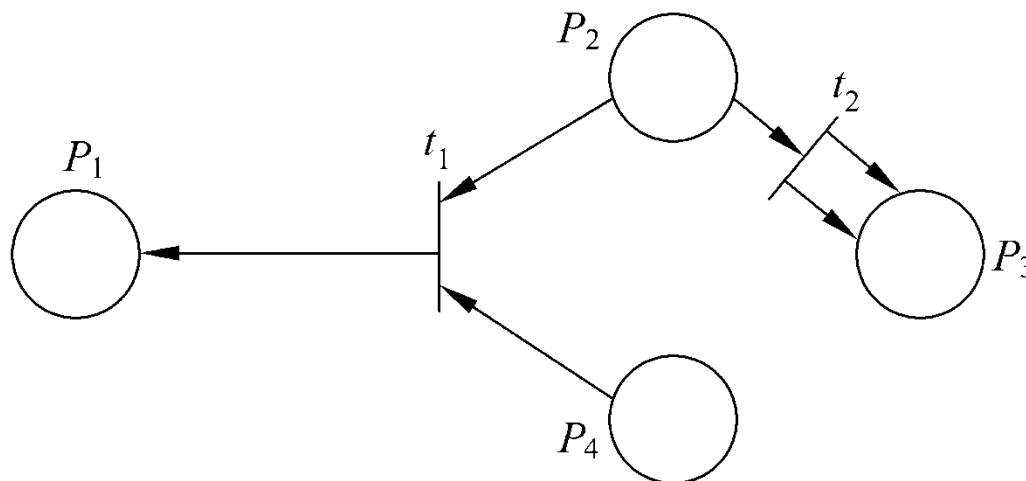
补充 3：形式化语言



● 形式化方法 2：Petri 网

● Petri 网包含 4 种元素：

- 一组位置 P 、
- 一组转换 T 、
- 输入函数 I 、
- 输出函数 O 。



● 其中：

- 一组位置 P 为 $\{P_1, P_2, P_3, P_4\}$ ，在图中用圆圈代表位置。
- 一组转换 T 为 $\{t_1, t_2\}$ ，在图中用短直线表示转换。

补充 3：形式化语言

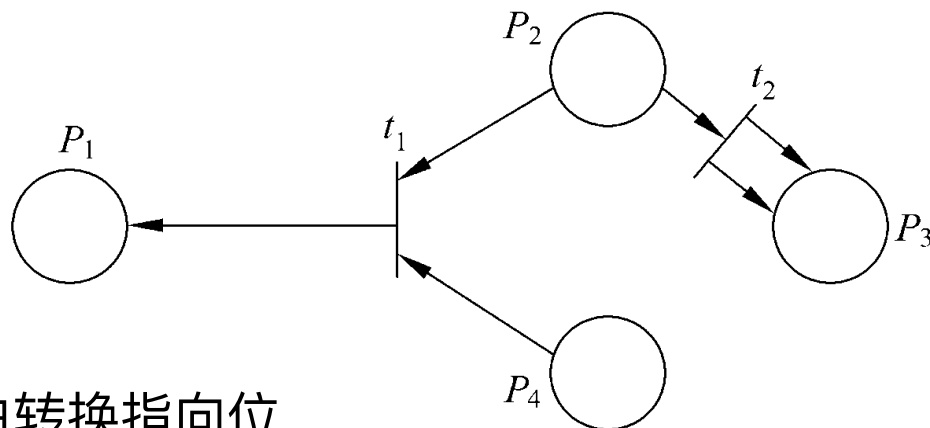


- 形式化方法 2：Petri 网

- 两个用于转换的输入函数，用由位置指向转换的箭头表示，是：

$$I(t_1) = \{P_2, P_4\}$$

$$I(t_2) = \{P_2\}$$



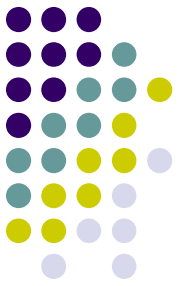
- 两个用于转换的输出函数，用由转换指向位置的箭头表示，是：

$$O(t_1) = \{P_1\}$$

$$O(t_2) = \{P_3, P_3\}$$

- 注意，输出函数 $O(t_2)$ 中有两个 P_3 ，是因为有两个箭头由 t_2 指向 P_3 。

补充 3：形式化语言



- 形式化方法 2：Petri 网

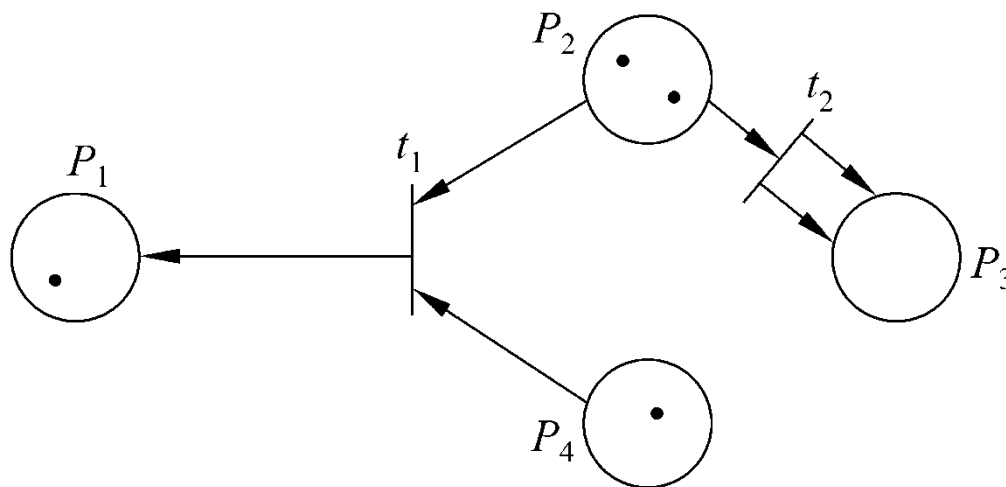
- 更形式化的 Petri 网结构，是一个四元组 $C=(P,T,I,O)$ 。
- 其中：
 - $P= \{ P_1 , \dots , P_n \}$ 是一个有穷位置集， $n \geq 0$ 。
 - $T= \{ t_1 , \dots , t_m \}$ 是一个有穷转换集， $m \geq 0$ ，且 T 和 P 不相交。
 - $I: T \rightarrow P^\infty$ 为输入函数，是由转换到位置无序单位组的映射。
 - $O: T \rightarrow P^\infty$ 为输出函数，是由转换到位置无序单位组的映射。

补充 3：形式化语言



● 形式化方法 2：Petri 网

- 一个无序单位组或多重组是允许一个元素有多个实例的广义集。
- Petri 网的标记是在 Petri 网中权标 (token) 的分配。例如，在下图中有 4 个权标，其中一个在 P_1 中，两个在 P_2 中， P_3 中没有，还有一个在 P_4 中。上述标记可以用向量 $(1, 2, 0, 1)$ 表示。由于 P_2 和 P_4 中有权标，因此 t_1 启动 (即被激发)。

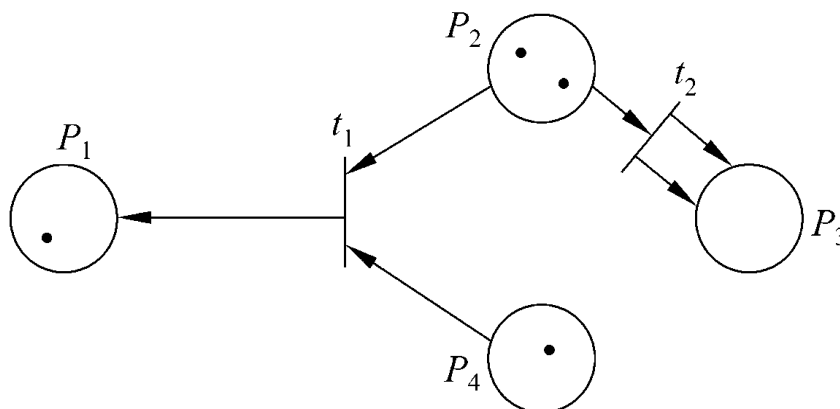


补充 3：形式化语言



● 形式化方法 2：Petri 网

- 通常，当每个输入位置所拥有的权标数大于等于从该位置到转换的线数时，就允许转换。当 t_1 被激发时， P_2 和 P_4 上各有一个权标被移出，而 P_1 上则增加一个权标。Petri 网中权标总数不是固定的，在这个例子中两个权标被移出，而 P_1 上只能增加一个权标。
- 在下图中 P_2 上有 2 个权标，因此除了激发 t_1 外， t_2 也可以被激发。当 t_2 被激发时， P_2 上将移走一个权标，而 P_3 上新增加两个权标。Petri 网具有非确定性，也就是说，如果数个转换都达到了激发条件，则其中任意一个都可以被激发

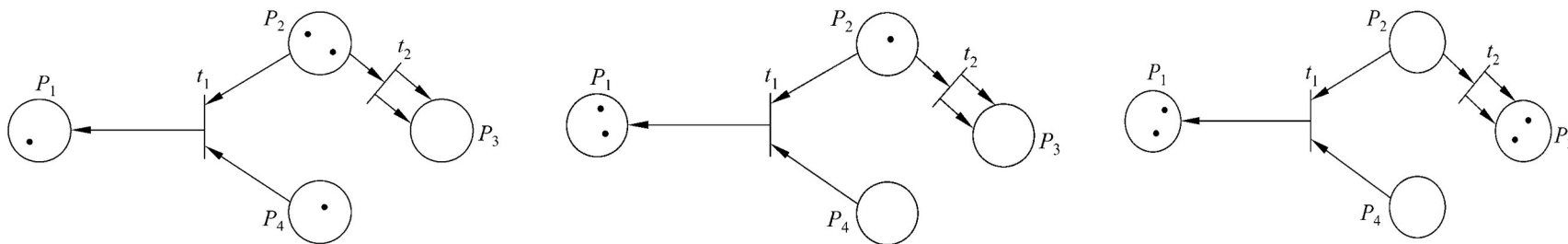


补充 3：形式化语言

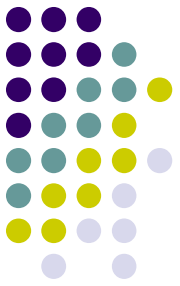


● 形式化方法 2：Petri 网

- 下图 1 所示 Petri 网的标记为 $(1, 2, 0, 1)$ ， t_1 和 t_2 都可以被激发。假设 t_1 被激发了，则结果如下图 2 所示，标记为 $(2, 1, 0, 0)$ 。此时，只有 t_2 可以被激发。如果 t_2 也被激发了，则权标从 P_2 中移出，两个新权标被放在 P_3 上，结果如下图 3 所示，标记为 $(2, 0, 2, 0)$ 。



补充 3：形式化语言

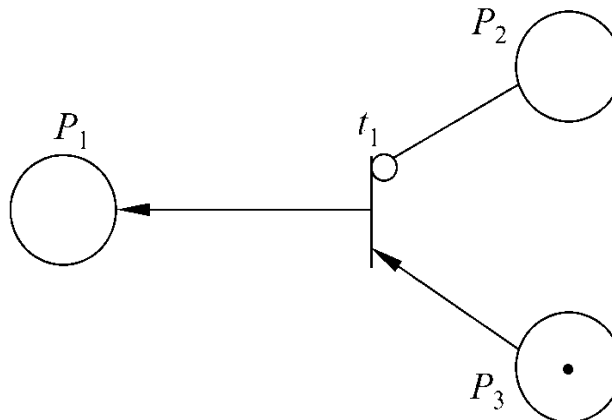


● 形式化方法 2：Petri 网

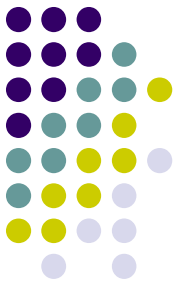
- 更形式化地说，Petri 网 $C=(P, T, I, O)$ 中的标记 M ，是由一组位置 P 到一组非负整数的映射：

$$M: P \rightarrow \{0, 1, 2, \dots\}$$

- 带有标记的 Petri 网成为一个五元组 (P, T, I, O, M) 。
- 对 Petri 网的一个重要扩充是加入禁止线。如图所示，禁止线是用一个小圆圈而不是用箭头标记的输入线。
 - 若一个位置 P 通过禁止线连接到转换，只有在该位置 P 中权标数量为零时，该转换才能被触发。



补充 3：形式化语言



- 形式化方法 2：Petri 网

- 现在把 Petri 网应用于上一节讨论过的电梯问题。当用 Petri 网表示电梯系统的规格说明时，每个楼层用一个位置 F_f 代表 ($1 \leq f \leq m$)，在 Petri 网中电梯是用一个权标代表的。在位置 F_f 上有权标，表示在楼层 f 上有电梯。

补充 3：形式化语言



● 形式化方法 2：Petri 网

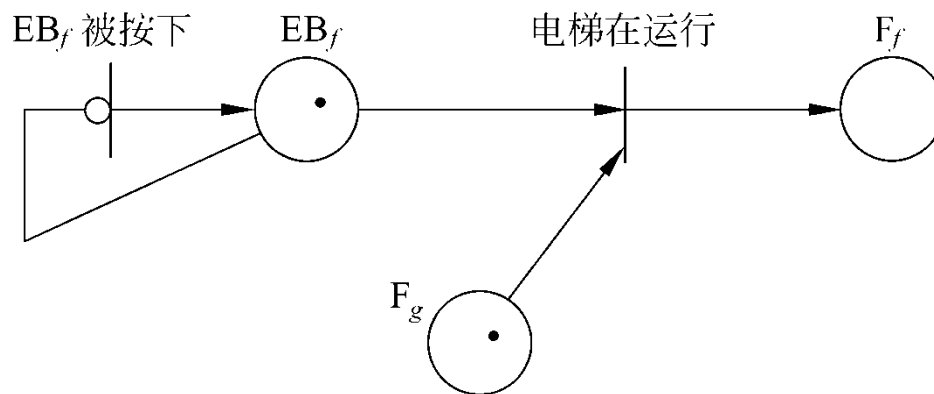
- 1. 电梯按钮
- 电梯问题的第一个约束条件描述了电梯按钮的行为。
- 第一条约束 C1：每部电梯有 m 个按钮，每层对应一个按钮。当按下一个按钮时该按钮指示灯亮，指示电梯移往相应的楼层。当电梯到达指定的楼层时，按钮将熄灭。
- 为了用 Petri 网表达电梯按钮的规格说明，在 Petri 网中还必须设置其他的位置。电梯中楼层 f 的按钮，在 Petri 网中用位置 EB_f 表示 ($1 \leq f \leq m$)。在 EB_f 上有一个权标，就表示电梯内楼层 f 的按钮被按下了。

补充 3：形式化语言



● 形式化方法 2：Petri 网

- 电梯按钮只有在第一次被按下时才会由暗变亮，以后再按它则只会被忽略。如图所示的 Petri 网描述了电梯按钮的行为规律。首先，假设按钮没有发亮，在位置 EB_f 上没有权标，在存在禁止线的情况下，转换“ EB_f 被按下”是允许发生的。现在按下按钮，则转换被激发并在 EB_f 上放置了一个权标
- 如图所示。因此，位置 EB_f 上的权标数不会多于 1。

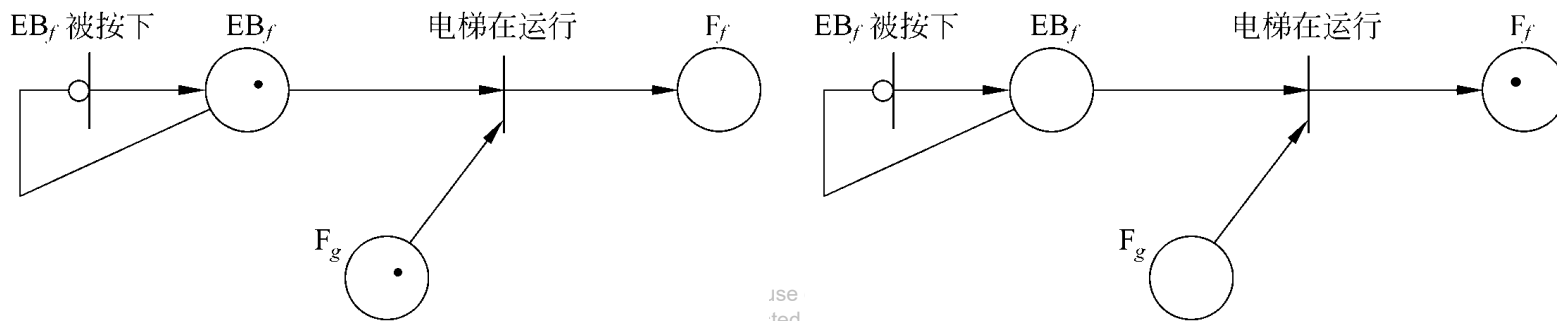


补充 3：形式化语言



● 形式化方法 2：Petri 网

- 假设电梯由 g 层驶向 f 层，因为电梯在 g 层，如图所示，位置 F_g 上有一个权标。由于每条输入线上各有一个权标，转换“电梯在运行”被激发，从而 EB_f 和 F_g 上的权标被移走，按钮 EB_f 被关闭，在位置 F_f 上出现一个新权标，即转换的激发使电梯由 g 层驶到 f 层。
- 事实上，电梯由 g 层移到 f 层是需要时间的，为处理这个情况及其他类似的问题，Petri 网模型中必须加入时限。也就是说，在标准 Petri 网中转换是瞬时完成的，而在现实情况下就需要时间控制 Petri 网，以使转换与非零时间相联系。



补充 3：形式化语言



● 形式化方法 2：Petri 网

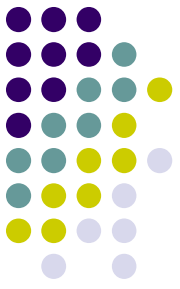
2. 楼层按钮

在第二个约束条件中描述了楼层按钮的行为。

第二条约束C2: 除了第一层与顶层之外，每个楼层都有两个按钮，一个要求电梯上行，另一个要求电梯下行。这些按钮在按下时发亮，当电梯到达该层并将向指定方向移动时，相应的按钮才会熄灭。

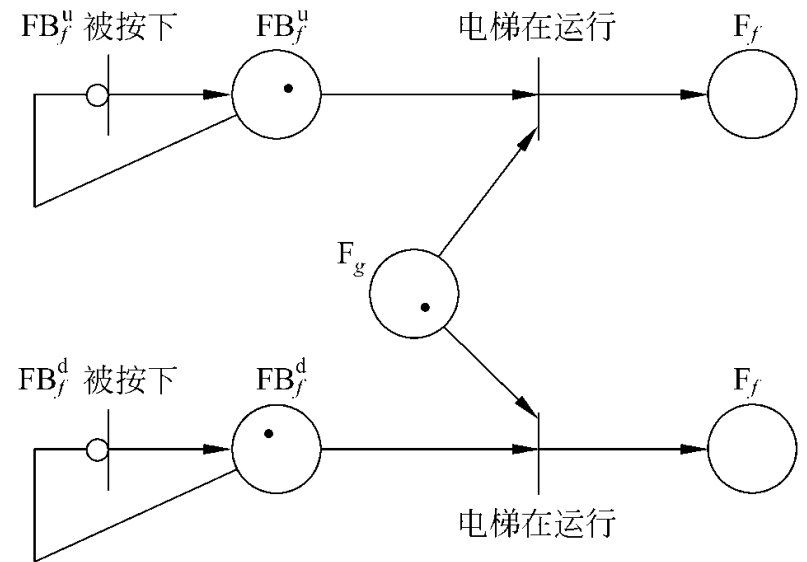
在Petri网中楼层按钮用位置 FB_f^u 和 FB_f^d 表示，分别代表f楼层请求电梯上行和下行的按钮。底层的按钮为 FB_1^u ，最高层的按钮为 FB_m^d ，中间每一层有两个按钮 FB_f^u 和 FB_f^d ($1 < f < m$)。

补充 3：形式化语言



● 形式化方法 2：Petri 网

- 图所示的情况为电梯由 g 层驶向 f 层。根据电梯乘客的要求，某一个楼层按钮亮或两个楼层按钮都亮。如果两个按钮都亮了，则只有一个按钮熄灭。
- 图所示的 Petri 网可以保证，当两个按钮都亮了的时候，只有一个按钮熄灭。但是要保证按钮熄灭正确，则需要更复杂的 Petri 网模型，对此不做更进一步的介绍。



补充 3：形式化语言



• 形式化方法 2：Petri 网

第三条约束C3：当电梯没有收到请求时，它将停留在当前楼层并关门。

这条约束很容易实现，如图4.11所示，当没有请求(FB_f^u 和 FB_f^d 上无权标)时，任何一个转换“电梯在运行”都不能被激发。

